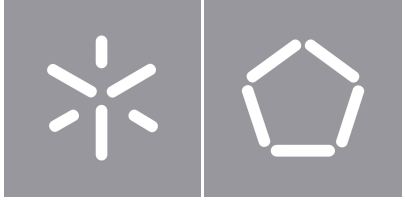


University of Minho
School of Engineering

Rui Filipe Pimenta Armada

Authoring Pedestrian Activities with Capacity-Aware Stations and Hybrid Boids

October 2026



University of Minho
School of Engineering

Rui Filipe Pimenta Armada

Authoring Pedestrian Activities with Capacity-Aware Stations and Hybrid Boids

Master's Dissertation in Informatics Engineering
Work supervised by
Professor Doctor António José Borba Ramires Fernandes

October 2026

Copyright and Terms of Use for Third Party Work

This dissertation reports on academic work that can be used by third parties as long as the internationally accepted standards and good practices are respected concerning copyright and related rights.

This work can thereafter be used under the terms established in the license below.

Readers needing authorization conditions not provided for in the indicated licensing should contact the author through the RepositóriUM of the University of Minho.

License granted to users of this work:



CC BY

https://creativecommons.org/licenses/by/4.0/

Acknowledgements

Agradeço ao meu orientador, Professor António José Borba Ramires Fernandes, pelo acompanhamento atento nesta última etapa académica e pela paciência, conhecimento, bom humor e exemplo profissional que me deu desde a licenciatura. Levo comigo a sua exigência e a sua paixão pela computação gráfica.

Às pessoas mais importantes da minha vida, obrigado. Ao meu pai, Rui, que sempre me apoiou e me mostrou o que é a integridade e força. À minha irmã, Carolina, a voz da razão que me acolheu nos momentos mais difíceis. E à minha mãe, Paula, o meu maior amor, que me trouxe calma e sempre foi a minha fã número um. Só cheguei aqui porque lutámos juntos, e esta conquista é tanto vossa como minha. Agradeço também aos familiares que acompanharam esta viagem, mas, se começasse a escrever nomes, nunca mais daqui saía.

Aos meus amigos, companheiros de batalha passados e futuros, obrigado por tudo o que me ensinaram. Ao Carlos António, o irmão que nunca tive desde o 2.º ano da licenciatura: que venham muitos mais anos. À Márcia Teixeira e à Sofia Marques, as minhas amigas lindas, que prezo mais do que ninguém: nunca saberei retribuir o que fizeram por mim. Agradecer à Bea, ao Miguel, à Filipa, à Morgana, ao Artur, ao Dinis, ao Rui e ao Zé. Sois todos uma bênção que levo no coração para sempre.

Dedico esta dissertação a cinco pessoas. Quatro já não estão entre nós, mas sinto as suas mãos nos meus ombros enquanto escrevo. À minha avó Lia, que continua ao meu lado e me ensinou a ser casmurro e a persistir. Ao meu avô Nelinho, que me inspirou a deixar sempre um bom nome, porque o mundo precisa de mais pessoas boas. Ao meu avô Pimenta, que me ensinou a ser ambicioso e a apontar para as estrelas, por mais impossível que pareça. À minha avó Dina, um dos grandes amores da minha vida, que me deu amor incondicional e cuja falta não se descreve por palavras. E à minha querida Zinda, a minha "criança grande", a estrelinha dos meus olhos, que me ensinou a viver com leveza, alegria e pela qual sinto falta todos os dias. Amo-vos aos cinco e daria tudo para estarmos todos juntos mais uma vez.

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I confirm that I have not used plagiarism or any form of undue use of information or falsification of results along the process leading to its elaboration.

I further declare that I have fully acknowledged the Code of Ethics and Conduct of the University of Minho.

Declaration on the Use of Artificial Intelligence (AI) Tools

I hereby declare having acted this academic work with integrity and confirm that I have not resorted to any form of undue use of AI tools, nor to the falsification or omission of information, sources, data, results or processes in any of the stages leading to its elaboration.

I further declare that any use of AI tools was done in a transparent and responsible manner, being duly noted in the work, and that I critically verified the accuracy of the content produced with their support, assuming full responsibility for what I submit.

I further declare that I have fully acknowledged the Code of Ethics and Conduct of the University of Minho, as well as the applicable standards of academic integrity and data protection.

AI tools were used in this work as follows: Claude and Grammarly helped with grammar, language revision and punctuation, and Claude also helped with the test and analysis scripts and with automating their runs.

University of Minho, Braga, October 2026

Rui Filipe Pimenta Armada

Resumo

Autoria de atividades pedonais com estações sensíveis à capacidade e boids híbridos

Muitas perguntas sobre edifícios e espaços públicos dependem do que as pessoas precisam, como uma secretária ou uma mesa, e do que acontece quando outra pessoa precisa dele ao mesmo tempo. Uma simulação que lhes responda tem de mostrar quando cada atividade começa de facto e porque se atrasa. Esta dissertação propõe uma estrutura em que horários em XML indicam quem faz o quê e quando, estações com capacidade limitada, fila e tempo de serviço decidem quando cada atividade ocorre, e uma camada de movimento híbrida segue caminhos de navegação, que os boids apenas desviam ligeiramente. Uma hora definida é, assim, um pedido, cujo início real depende do que a pessoa ainda faz, da distância e da disponibilidade do recurso, e um registo de regras, visitas e caminhadas explica cada atraso.

Implementámos a estrutura em C++ na Unreal Engine 5.4, com uma única implementação de pessoa numa rotina urbana diária com 50 pessoas, em dois centros comerciais, numa evacuação e em corredores de fluxo dirigido, com apenas uma ação nova em todos os cenários. Com a marcha e o relógio sincronizados, a avaliação mediu a origem da diferença entre os tempos definidos e os realizados. A espera por estações partilhadas dominou essa diferença nos centros comerciais, onde as filas foram 76% do atraso no início das regras de estação e 92% numa hora de almoço com um lugar por balcão, enquanto na rotina diária pesaram mais as caminhadas longas até destinos congestionados. As mudanças de capacidade comportaram-se como uma sequência de filas: em 30 execuções por configuração, um segundo ou terceiro lugar nas mesas reduziu a mediana da espera por visita de 0,84 para 0,51 minutos simulados, enquanto mais lugares nos balcões apenas deslocaram a fila para as mesas, como se viu na hora de almoço, com esperas de vários minutos. Estas tendências mantiveram-se com dois passos, mas não a sua dimensão. Os boids espalharam os grupos em dois de três corredores e aliviaram um estrangulamento, e um só computador anima 50 a 100 pessoas com uma média acima de 30 imagens por segundo, embora as mais lentas fiquem abaixo desse valor. Sem validação com peões reais, os resultados descrevem o modelo, não edifícios reais.

Palavras-chave Autoria de regras, Boids, Recursos partilhados, Simulação de multidões, Unreal Engine

Abstract

Authoring Pedestrian Activities with Capacity-Aware Stations and Hybrid Boids

Many questions about buildings and public spaces are about what people need, such as a desk or a table, and what happens when someone else needs it at the same time. A simulation that answers them must show when each activity actually starts and why it starts later than planned. This thesis presents a rule-authoring framework for pedestrian activities, in which XML schedules state who does what and from when, stations with a limited capacity, a first-come, first-served queue and a service time decide when each activity can take place, and a hybrid movement layer follows navigation paths while the boids forces only turn the heading slightly. An authored time is therefore a request, whose actual start depends on what the person is still doing, how far they must walk and whether the resource is free, and a recorder logs every rule, visit and walk to explain each delay.

We built the framework in C++ in Unreal Engine 5.4 and used one implementation of a person for a daily urban routine with 50 people, two shopping center layouts, an emergency evacuation and a set of directed-flow corridors, with only one new action across all scenarios. With walking and the clock advancing together, the evaluation measured where the gap between authored and realized times comes from. Waiting for shared stations was the largest part of that gap in the shopping centers, where queueing made up 76% of the start delay of station rules and 92% in a lunch rush with one place per counter, while in the daily routine long walks to crowded target points took a larger share. Capacity changes behaved as a pipeline of queues: over 30 runs per setting, a second or third place at the food tables lowered the median wait per station visit from 0.84 to 0.51 simulated minutes, while extra places at the food counters only moved the queue to the tables, and the lunch rush showed the same shift with waits of several minutes. These directions held at two frame steps, although their magnitudes varied. The boids blend spread the cohorts in two of three test corridors and eased a bottleneck, and one desktop computer animates 50 to 100 people at a mean above 30 frames per second, although its slowest frames fall below that. Without validation against real pedestrians, the results describe the model rather than real buildings.

Keywords Boids, crowd simulation, rule authoring, shared resources, Unreal Engine

Contents

I Introductory Material 1

1 Introduction 2

1.1 Motivation and Context . . . 3

1.1.1 Crowds as an activity problem . . . 3

1.1.2 From flocking demonstrations to authored studies . . . 3

1.2 Main Goals . . . 4

1.2.1 Research question . . . 5

1.2.2 Objectives . . . 5

1.3 Document Structure . . . 6

2 Boids, Agent-Based Models and Crowd Simulation 7

2.1 Boids and Steering . . . 7

2.1.1 Combining behaviors . . . 9

2.1.2 Cost of neighbor search . . . 10

2.2 Agent-Based Modeling . . . 10

2.3 Action Selection and Crowd Models . . . 11

2.4 Purposeful Populations and Shared Resources . . . 14

2.5 Discussion . . . 16

3 Scheduled Pedestrians and Shared Resources 17

3.1 Introduction . . . 17

3.2 Behavior . . . 17

3.2.1 Overview of the approach . . . 18

3.2.2 Functional requirements . . . 19

3.3 Issues . . . 19

3.3.1 Purpose versus motion . . . 19
3.3.2 Timing under contention . . . 20
3.3.3 Shared resources and queues . . . 21
3.3.4 Reuse across scenarios . . . 21
3.3.5 Measurement and performance . . . 21

II Core of the Dissertation 22

4 A Rule-Authoring Framework for Pedestrian Activity 23

4.1 Introduction . . . 23
4.2 First Approach . . . 24
4.3 Pedestrian Movement Simulator Architecture . . . 24
4.3.1 Architecture and module layout . . . 24
4.3.2 Schedules and the rule language . . . 26
4.3.3 Actions . . . 29
4.3.4 Movement: capsules, probes and the hybrid blend . . . 30
4.3.5 Stations, areas, super stations and pipelines . . . 31
4.3.6 Environment and scenario managers . . . 33
4.3.7 Simulation clock and run modes . . . 34
4.3.8 Configuration and instrumentation . . . 35
4.3.9 Extending the framework . . . 36
4.4 Summary . . . 37

5 Evaluation Based on Use Cases 38

5.1 Introduction . . . 38
5.2 Experimental Setup . . . 38
5.2.1 Machine, modes and settings . . . 39
5.2.2 What was recorded . . . 40
5.3 Daily Urban Life . . . 41
5.4 Shopping Center: Mall A and Mall B . . . 43
5.4.1 Capacity experiments . . . 45
5.4.2 A lunch rush . . . 46
5.4.3 Where late minutes come from . . . 47

5.4.4 Mall A against Mall B . . . 47
5.5 Emergency Evacuation . . . 48
5.6 Directed-Flow Test Corridors . . . 50
5.6.1 The hybrid blend on and off . . . 50
5.7 Performance and System Scalability on One Machine . . . 51
5.8 Threats to Validity . . . 53
5.8.1 Edge cases and their effect on each result . . . 54
5.8.2 Frame step . . . 54
5.9 Summary . . . 55

6 Conclusions and Future Work **56**
6.1 Conclusions . . . 56
6.1.1 Objectives and requirements . . . 57
6.1.2 Limitations . . . 57
6.2 Prospect for Future Work . . . 58
6.2.1 Other populations and places . . . 58
6.2.2 Performance . . . 59
6.2.3 Shared resources and timing . . . 59
6.2.4 Calibration and validation . . . 59
6.2.5 Authoring tools . . . 59

III Appendices **69**

A Details of Results **70**
A.1 Shopping Center . . . 70
A.2 Start Delay . . . 71
A.3 Evacuation . . . 71
A.4 Scaling . . . 72

B Tooling **74**
B.1 Building and Running . . . 74
B.2 Scaling Schedules, Analysis and Documents . . . 74
B.3 Repeating the Runs . . . 75

Acronyms

ABM agent-based model.

CSV comma-separated values.

FIFO first in, first out.

FPS frames per second.

FSM finite-state machine.

GPU graphics processing unit.

ODD Overview, Design concepts, and Details.

ORCA optimal reciprocal collision avoidance.

RVO reciprocal velocity obstacles.

UE Unreal Engine.

XML Extensible Markup Language.

List of Figures

1 A food court 4

2 Collective motion in nature 7
3 The three boids rules 8
4 Steering between action selection and locomotion 9
5 A two-dimensional k-d tree 10
6 A behavior tree 11
7 A navigation mesh built by Recast 12
8 Movement on a grid and in continuous space 13
9 A velocity obstacle 13
10 A taxonomy of crowd motion models 13
11 A bottleneck experiment in a virtual environment 16

12 Timeline of one rule that uses a shared resource 18
13 Overview of the approach 19
14 A queueing node 21

15 Classes grouped by layer 25
16 Geometry of the hybrid blend 31
17 Lifecycle of one station step 33

18 Daily Urban Life map during the morning 42
19 Walks that arrived at each work area 43
20 Mall A 44
21 Mall B 44
22 Median wait per visit by number of places 45
23 Start delay of station rules 47

24 Evacuation shortly after the alarm 49
25 People who passed an exit 49
26 Frame rate and memory by number of people 52
27 Frame rate during one run with 400 people 53

List of Tables

1 Systems that give simulated populations purpose 15

2 Functional requirements 20

3 Requirements and where they are addressed 23

4 Elements of the schedule language 26

5 Reasons for deferring a due rule 28

6 The six schedule actions 29

7 Priority layers of the person 31

8 Run recorder files 36

9 Settings changed for the experiments 37

10 Daily Urban Life outcomes 42

11 Mall A during a lunch rush 46

12 Hybrid blend on and off 51

13 Frame time with and without the characters 52

14 Edge cases and their effect on each result 54

15 Assessment of the requirements 58

A.1 Wait per visit for every capacity run 70

A.2 Wait per visit by number of places, 30 runs 71

A.3 Start delay of station rules 71

A.4 Evacuation per run 72

A.5 Scaling runs 73

B.1 Command-line flags 75

Part I

Introductory Material

Chapter 1

Introduction

Pedestrian simulations allow us to ask questions about a place before it is built or altered. Will the food court have enough tables during the lunch rush, will two shifts of office workers compete for the same desks, and will one of the exits become a choke point when the alarm sounds? The way people walk gives part of the answer, but the rest depends on why they move, what they are looking for when they get there and how the time spent waiting for one thing delays the next.

This dissertation studies how a simulation of purposeful pedestrians can show where and why the activities in a place start later than intended. Boids are well known for the emergent collective motion that results from a small number of local rules (Reynolds, 1987), and they are straightforward to incorporate into a game engine. The basic model, however, has nothing that would send one boid to work at 9 am or make another need a table at the food court. We developed a framework that treats purpose as data, so that the analysis of a place, that is, when each activity actually starts, how long people wait for shared resources and how a change in capacity affects the delays, can be extracted from recorded runs. To make such analyses cheap, the system is designed to describe different places with as little change to its code as possible, so a new study requires new schedules, maps and settings rather than new code. Schedules define who does what and when, a movement layer combines navigation paths with the bounded steering of the boids, stations with a finite capacity determine when an activity can start, and the environment constrains where people can go. We implemented the framework in **Unreal Engine (UE)** 5.4 with public source code, schedules and maps (Armada, 2026), and the same implementation of a person simulates a daily urban routine, a shopping center, an emergency evacuation and a set of directed-flow corridors.

This chapter presents the motivation (Section 1.1), the research question and the objectives (Section 1.2), and the structure of the document (Section 1.3).

1.1 Motivation and Context

1.1.1 Crowds as an activity problem

People in a station, a shopping center or an office are rarely just wandering about, as they are almost always on their way to buy a ticket, sit at a desk, have a meal or leave the building. Research into pedestrian behavior identifies three levels of decision: the strategic level, which plans the activities, the tactical level, which decides where to do them and which route to take, and the operational level, which is the walking itself and the avoidance of other people (Hoogendoorn and Bovy, 2004).

Models of local motion work only at the operational level, as boids use separation, alignment and cohesion (Reynolds, 1987), and the social force model moves people toward their desired velocity while other people and walls push them away (Helbing and Molnár, 1995). Reciprocal velocity models restrict each individual to the velocities that avoid a collision with its neighbors (van den Berg et al., 2011). These models describe what happens once people have decided where to go, but they do not say what the destination is or what happens when the resource that a person needs is not available. Other work looks at the level of activities above locomotion. Autonomous pedestrians buy a ticket, queue and rest in a virtual train station (Shao and Terzopoulos, 2007), the people of Paris and Donikian (2009) carry out goal-oriented daily activities, and at the scale of a city, MATSim produces daily plans that link activities through trips (Horni et al., 2016).

Waiting depends on both capacity and demand: a food court (Fig. 1), for example, has a fixed number of counters and tables, so when demand exceeds capacity, people wait and their next activity is delayed. Queueing theory treats such resources as servers with a given capacity, an arrival process and a service time (Kendall, 1953; Shortle et al., 2018). In a crowd simulation, the arrival process is derived from the individual schedules, the walking times and the earlier delays. A simulation that studies capacity must therefore model activities, resources and durations, and it must let waiting arise from these rather than impose it.

1.1.2 From flocking demonstrations to authored studies

Reynolds breaks character behavior down into three layers: action selection, steering and locomotion (Reynolds, 1999). Boids operate at the steering level, and a flocking demonstration needs little action selection, but as soon as a demonstration takes place in a particular location, action selection becomes the dominant issue. When each demonstration has its own controller, the code has to be rewritten for

every combination of questions, such as a morning routine with fewer desks than workers or the same routine with an alarm going off. It is then difficult to tell whether the differences between runs come from the change under study or from the code written for that particular scenario.



Figure 1: A food court, whose tables are shared by everyone who wants to eat at the same time (photo by BornToZucc, cropped, CC BY-SA 4.0, via Wikimedia Commons).

Several lines of research take this knowledge out of the person. CAROSA gives virtual characters roles, schedules and parameterized actions (Allbeck, 2010), and functional crowds follow roles and schedules in a building (Pelechano et al., 2008). Smart objects contain the information necessary to interact with them (Kallmann and Thalmann, 1999), characters in The Sims choose objects that advertise what they offer (Forbus and Wright, 2001), and the Smart Objects of Unreal Engine let people reserve a slot (Epic Games, 2026e). Menge breaks a crowd simulation down into components that can be swapped (Curtis et al., 2016), and the ODD protocol asks modelers to report the entities and the scheduling of their model (Grimm et al., 2020).

Our approach builds on these principles, so a person who studies a location should be able to specify the demand with explicit rules and state which resources the environment provides, without having to be a programmer. A new study should also leave the implementation of a person unchanged. Our central idea is that a stated time is a request, that is, a rule that defines when an activity may start, while the simulation decides when it actually starts.

1.2 Main Goals

The aim is to measure how shared resources affect the realized timetable of a place, so that a user can see where and why activities start later than authored. Authored rules, rather than code written for each

individual case, give a population of boids purpose, a timetable and a range of activities, and the same system serves every study.

1.2.1 Research question

> *How can authored rules over capacity-limited stations, followed by a reusable population of boids, show where and why the realized timing of the activities in a place departs from the authored one?*

The question has a measurement part and an authoring part. The measurement part asks what the recorded runs tell us about waiting, walking and capacity in each place, and the authoring part asks how few rule types and how little scenario code each study needs, as a reusable population keeps the class that represents a person, the movement layer and the resource model the same in every scenario. The scenarios test both parts, because they require routines, shared resources and emergencies to be written in the same way and then measured.

1.2.2 Objectives

Chapter 3 sets out the functional requirements R1 to R7, and Chapter 6 assesses each of the following objectives against them.

01: A language for purpose and timing, which is an **Extensible Markup Language (XML)** schedule format that states which person acts, when, with what action and on what target, with explicit rules for deferring, dispatching and dropping rules (R1, R2).

02: A movement layer that takes each person to their target by combining navigation mesh path following with boids steering, while the environment, with its walking surfaces, walls, gates and exits, limits movement without being part of the rules (R5).

03: Shared resources with a limited capacity, which are stations with places, a first-come, first-served queue and a service time, grouped into areas and used in ordered activities with several stages (R2, R3).

04: Reuse across a range of scenarios, with several scenario families in **UE** 5.4 built on a single implementation of a person, to find where reuse breaks down (R4).

05: Measurement and a preliminary assessment, which use records of every rule, station visit and trip to show how capacity, travel and population affect the realized timing, and which measure how well the implementation scales on one machine (R6, R7).

1.3 Document Structure

Part I contains this introduction, Chapter 2 (Boids, Agent-Based Models and Crowd Simulation) and Chapter 3 (Scheduled Pedestrians and Shared Resources). Chapter 2 covers boids and steering, agent-based modeling, action selection and crowd models, and the work on purposeful populations, queueing, evacuation and validation. Chapter 3 describes the behavior we want, gives a high-level overview of our approach without technical detail, sets out the functional requirements R1 to R7 and discusses the difficulties in meeting them.

Part II, the core of the dissertation, starts with Chapter 4, which presents the framework first through the approach built during the pre-dissertation work and then through the architecture of the current pedestrian movement simulator. Chapter 5 evaluates the system on a daily urban routine, two shopping center layouts, an evacuation and a set of directed-flow corridors, and Chapter 6 answers the research question and describes the future work. The appendices contain the per-run result tables (Appendix A) and the build and analysis tooling (Appendix B).

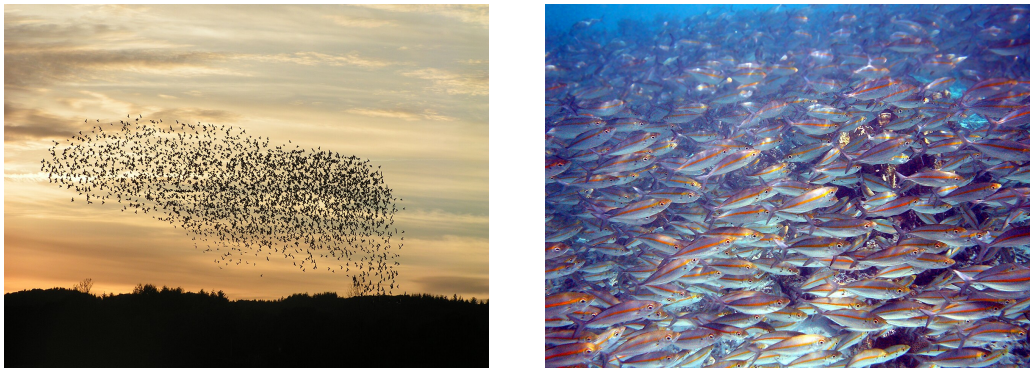
Chapter 2

Boids, Agent-Based Models and Crowd Simulation

The chapter starts with boids, steering and the cost of neighbor search (Section 2.1) and places boids in the context of agent-based modeling (Section 2.2). It then reviews action selection, the tools of the game engine and the crowd motion models (Section 2.3), and it concludes with the work on purposeful populations, queueing, evacuation and validation that is most relevant to this thesis, together with the gap that the thesis addresses (Sections 2.4 and 2.5).

2.1 Boids and Steering

Reynolds introduced boids in 1987 as a distributed model for flocks, herds and schools (Fig. 2). Instead of following a global description, each boid senses its immediate neighborhood and acts according to a small set of local rules, and the behavior of the flock as a whole emerges from these individual actions. Vicsek et al. (1995) subsequently demonstrated that particles which only respond to their neighbors can move from disorder to order even in the presence of noise. We retained boids because the project began from a boid prototype (Section 4.2) and because they are easy to plug into a game engine and have intuitive parameters, while the authoring layer of Chapter 4 does not depend on that choice.



(a) Starlings over Ørnsø, Denmark

(b) A school of *Pterocaesio chrysozona*

Figure 2: Collective motion in nature, with (a) starlings and (b) a fish school (photos by (a) Christoffer A. Rasmussen, public domain, and (b) Brocken Inaglory, CC BY-SA 3.0, via Wikimedia Commons).

A boid is a vehicle that behaves as a point with a position $\mathbf{p}_i$ and a velocity $\mathbf{v}_i$, and it faces the direction of its velocity. At each update, it sees the set N_i of flock members within a distance r and a field of view θ measured from its direction of travel (Reynolds, 1987):

$$N_i = \left\{ j \neq i \;:\; \|\mathbf{p}_j - \mathbf{p}_i\| < r \;\wedge\; \angle(\mathbf{v}_i,\, \mathbf{p}_j - \mathbf{p}_i) < \tfrac{\theta}{2} \right\} \tag{2.1}$$

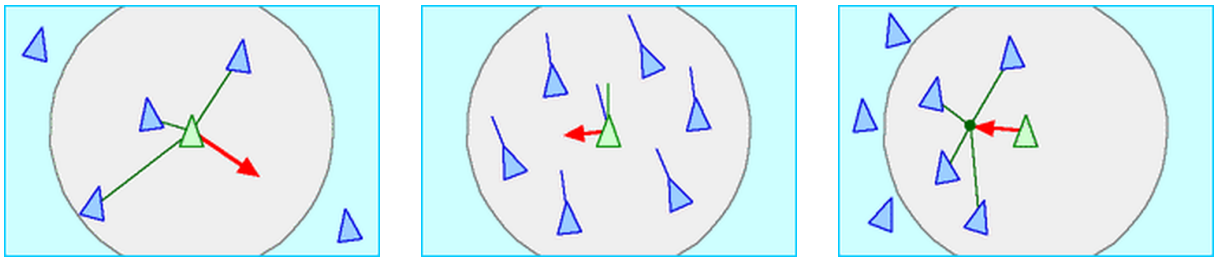
Three rules govern this neighborhood, which are to avoid collision, to match velocity and to stay close (Fig. 3). Reynolds put them into words, and the equations below are a typical way of expressing them, each giving zero when N_i is empty. Separation sums the vectors that point away from each neighbor, weighted by the inverse square of the distance, alignment matches the mean velocity of the neighbors, and cohesion pulls the boid toward their centroid:

$$\mathbf{s}_i = \sum_{j \in N_i} \frac{\mathbf{p}_i - \mathbf{p}_j}{\|\mathbf{p}_i - \mathbf{p}_j\|^2} \tag{2.2}$$

$$\mathbf{a}_i = \frac{1}{|N_i|} \sum_{j \in N_i} \mathbf{v}_j \;-\; \mathbf{v}_i \tag{2.3}$$

$$\mathbf{c}_i = \frac{1}{|N_i|} \sum_{j \in N_i} \mathbf{p}_j \;-\; \mathbf{p}_i \tag{2.4}$$

Separation is the only rule that keeps boids apart, and it is a weak constraint, because proximity only incurs a cost and boids can still overlap when other forces are stronger. The centroid is local, so a flock can split around an obstacle and rejoin. Strangers on a street have no reason to stay together, so for pedestrians cohesion is generally low, present only within a group, and overridden by each person's destination.



(a) Separation (b) Alignment (c) Cohesion

Figure 3: The three boids rules seen from the green boid, where the grey disc is its neighborhood and the red arrow is the steering that each rule produces (diagrams by Craig Reynolds, public domain, via Wikimedia Commons).

Reynolds (1999) built a library of steering behaviors on these rules and split movement into *action selection* (choosing a goal), *steering* (turning the goal into a desired change in motion) and *locomotion* (the movement itself, Fig. 4). In our design, authored rules take care of action selection and the boids-based navigation takes care of steering. The vehicle is a point mass m with a maximum force f_{max} and a

maximum speed $v_{\max}$:

$$\boxed{\mathbf{v} \leftarrow \text{trunc}\left(\mathbf{v} + \frac{\text{trunc}(\mathbf{f}, f_{\max})}{m}\,\Delta t,\ v_{\max}\right) \qquad \mathbf{p} \leftarrow \mathbf{p} + \mathbf{v}\,\Delta t} \tag{2.5}$$

Most behaviors compute a desired velocity $\mathbf{v}^d$ and set $\mathbf{f}$ to $\mathbf{v}^d$ minus $\mathbf{v}$, as in seek and flee, arrival, pursuit and evasion, wander, path following, obstacle avoidance with probes in front of the individual, and leader following.

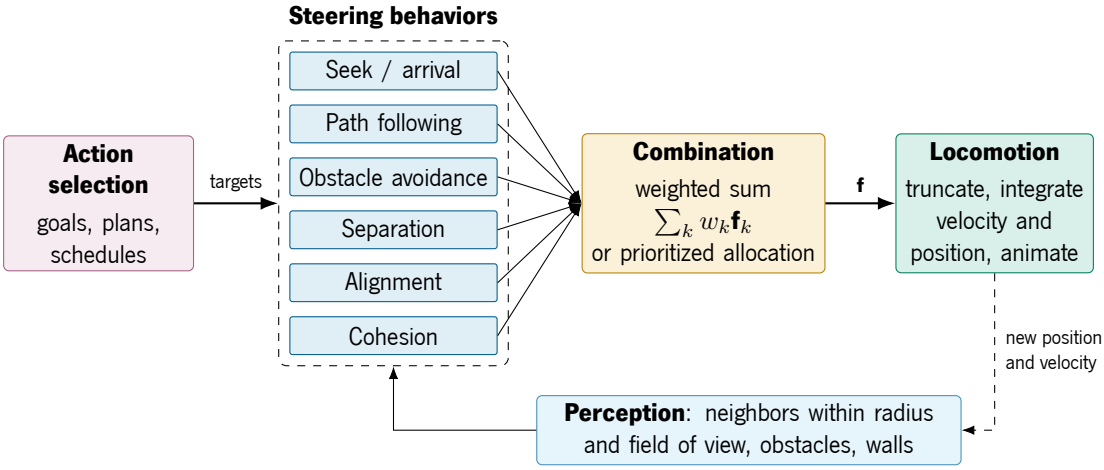


Figure 4: Steering behaviors between action selection and locomotion, after Reynolds (1999).

2.1.1 Combining behaviors

A pedestrian that follows a path must also keep clear of other pedestrians and of walls, and the weighted sum is the simplest way of combining the forces, as Reynolds (1999) noted:

$$\boxed{\mathbf{f} = \text{trunc}\left(\sum_k w_k\,\mathbf{f}_k,\ f_{\max}\right)} \tag{2.6}$$

It is cheap and gives smooth motion, but opposing forces can cancel each other out and leave the character stuck, a small force can mask a larger one, and weights that work in one situation may fail in another (Millington, 2019). The alternative is priority, as in Reynolds' prioritized acceleration allocation, where requests fill an accumulator in order of priority until it is full, and textbooks also blend within priority groups (Millington, 2019). Many systems, ours included, use priority between layers and blending within a layer. Extensions of the original rules add forces for leadership changes (Hartman and Beneš, 2006), fear signals that reach animals which cannot see the threat (Delgado-Mata et al., 2007) and escape, which in a building means competing for exits, where pushing makes everyone slower (Helbing et al., 2000).

2.1.2 Cost of neighbor search

In each frame, each boid evaluates Eqs. (2.1) to (2.4) from a snapshot of the previous state, combines the forces with Eq. (2.6) and integrates them with Eq. (2.5), so the result does not depend on the order of updates and the algorithm parallelizes well. A direct search performs $O(n^2)$ distance tests per frame, which is the main bottleneck with thousands of individuals. The usual remedy is a uniform grid whose cells are at least as large as the perception radius, so that a query visits only the neighboring cells and its cost depends on the local density (Ericson, 2004; Reynolds, 2006). Hierarchical structures such as k-d trees (Bentley, 1975) adapt better to clustered individuals but cost more to rebuild every frame (Fig. 5). General-purpose **graphics processing unit (GPU)** computation suits boids, because each individual only reads the previous state (Owens et al., 2007; Joselli et al., 2009), and approximate or capped neighbor sets reduce the cost further (Joselli et al., 2009; da Silva et al., 2009), although these change the model and should be reported.

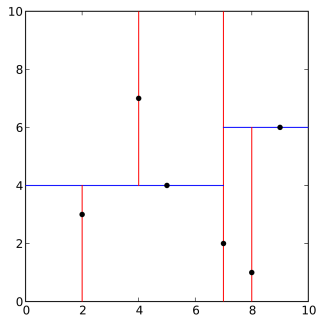


Figure 5: A two-dimensional k-d tree for seven points (image by KiwiSunset, English Wikipedia, CC BY-SA 3.0, via Wikimedia Commons).

2.2 Agent-Based Modeling

An **agent-based model (ABM)** represents a system as a collection of autonomous individuals, each of which assesses its own situation and acts according to a set of rules (Bonabeau, 2002), and boids are a classic example. Macal and North (2010) describe a model through individuals with attributes and behaviors, their interactions and their environment. Toolkits such as NetLogo (Wilensky, 1999), MASON (Luke et al., 2005) and Repast Simphony (North et al., 2013) target general modeling rather than animated characters on navigation meshes, which is one reason why interactive crowds are built inside game engines. Outcomes are usually emergent, as with lane formation in counter-flows (Helbing et al., 2005), so modelers run the model many times with different seeds and summarize the results statistically. The

ODD protocol (Grimm et al., 2006, 2020) also asks modelers to report hidden design decisions, such as the order of updates and how time advances.

A population can be studied top-down through aggregate equations, as when Hughes (2002) treats the crowd as a fluid moving down a potential, or bottom-up through individuals. Van Dyke Parunak et al. (1998) found that agent-based models are appropriate for systems of discrete decision makers, each with its own history, while equation-based models are appropriate for systems that aggregates describe well. Corridor flow obeys aggregate relations, but whether a worker finds a free desk depends on what other people did that morning, which is the kind of question this thesis is concerned with.

2.3 Action Selection and Crowd Models

A **finite-state machine (FSM)** has one state at a time and changes state when a condition is met, so the number of transitions grows quickly as states are added (Millington, 2019). Behavior trees, popularized by Halo 2 (Isla, 2005), compose sequences and selectors from leaves and are more modular and reactive (Millington, 2019; Colledanchise and Ögren, 2018) (Fig. 6). Goal-oriented action planning searches for actions whose preconditions and effects reach a goal (Orkin, 2006), and utility-based selection scores the options (Graham, 2019). These describe what to do rather than when to do it, so time of day and resource availability have to be modeled separately. Other work moves knowledge into the environment, with smart objects that describe how they are used (Kallmann and Thalmann, 1999), objects in The Sims that advertise how much they satisfy a motivation (Forbus and Wright, 2001) and behaviors attached to regions (Sung et al., 2004), and our stations are derived from these concepts. Data-driven and learned approaches reuse recorded trajectories (Lerner et al., 2007) or train individual policies (Juliani et al., 2018), such as collision avoidance (Lee et al., 2018), but they address local motion, and a user who must state who arrives when gains little from a policy that hides its assumptions in weights.

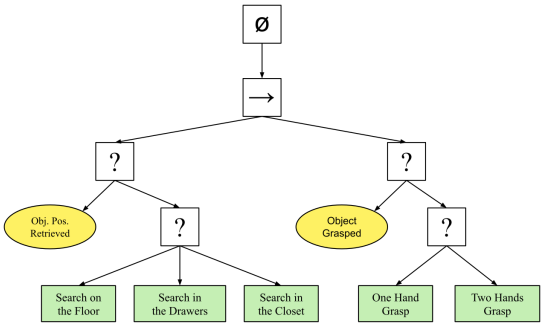


Figure 6: A behavior tree for a robot that finds and grasps an object (Colledanchise and Ögren, 2018) (diagram by Aliekor, English Wikipedia, CC BY-SA 3.0, via Wikimedia Commons).

Game engines now provide much of this infrastructure, and we draw on the current Unreal Engine documentation for its concepts, although the implementation uses version 5.4. The Navigation System builds a navigation mesh with Recast and answers path queries (Epic Games, 2026d; Mononen, 2009) (Fig. 7), with **reciprocal velocity obstacles (RVO)** avoidance in the Character Movement component and adaptive **RVO** sampling in the Detour Crowd manager (Epic Games, 2026a). Behavior trees and State Trees support decisions (Epic Games, 2026f), Smart Objects have slots that characters claim and release (Epic Games, 2026e), and Mass Entity, the Mass Gameplay plugins and ZoneGraph lanes support large numbers of characters (Epic Games, 2026b,c,g). Who does what and when, the order of waiting and the service time are all left to the developer. Unlike Smart Objects, which let characters claim and release slots, our stations add a capacity, a first-come, first-served queue, a timed service and ordered multi-stage use, and our schedules add the timing and deferral rules that decide when each request is made. Menge makes goal selection, planning and local avoidance pluggable components (Curtis et al., 2016), which is also our motivation at the boundary between activity and movement.

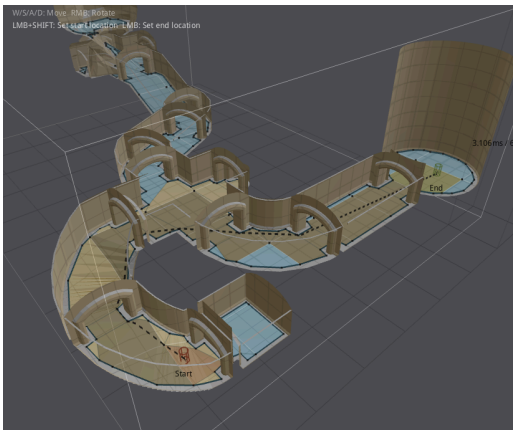


Figure 7: A navigation mesh built by Recast, with a path found by Detour (cropped screenshot of RecastDemo, Recast & Detour © 2009 Mikko Mononen, zlib license, github.com/recastnavigation).

General references on crowd simulation are Thalmann and Musse (2013), Pelechano et al. (2008) and Kapadia et al. (2016), with a history by Musse et al. (2021), and the motion models below all take their targets from an action selection layer (Fig. 10). Macroscopic models evolve a density field over a cost potential (Hughes, 2002; Treuille et al., 2006), and cellular automata with floor fields (Burstedde et al., 2001; Kirchner and Schadschneider, 2002) are fast but too coarse for animated motion (Fig. 8). Microscopic models compute motion per pedestrian with social forces (Helbing and Molnár, 1995), HiDAC (Pelechano et al., 2007), reciprocal velocity obstacles and **optimal reciprocal collision avoidance (ORCA)** (van den Berg et al., 2008, 2011) (Fig. 9), a time-to-collision law (Karamouzas et al., 2014) or vision-based heuristics (Moussaïd et al., 2011), as surveyed by van Toll and Pettré (2021). Force-based

models and boids only penalize overlap, velocity obstacle models avoid it under their own assumptions, and the walker experiments of Pettré et al. (2009) show that both people share the avoidance. None of these models provides targets, so the authoring problem would be the same with any of them.

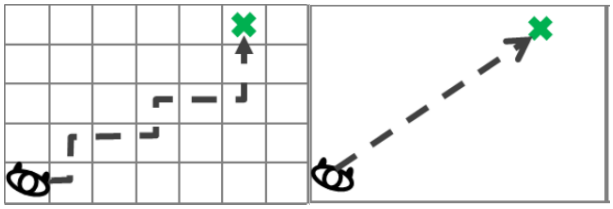


Figure 8: Movement on a grid of cells, on the left, and in continuous space, on the right (diagram by Angelika Kneidl, CC BY-SA 4.0, via Wikimedia Commons).

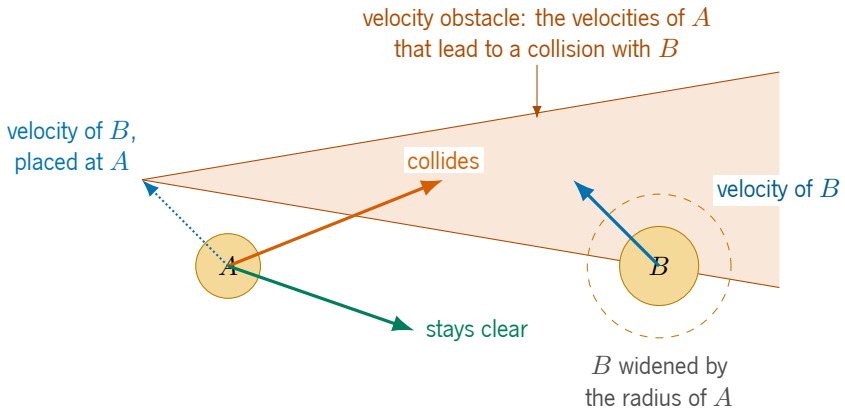


Figure 9: A velocity obstacle, where every velocity of A that ends inside the shaded cone leads to a collision with B if both keep moving, so velocity obstacle models choose a velocity outside it (van den Berg et al., 2008).

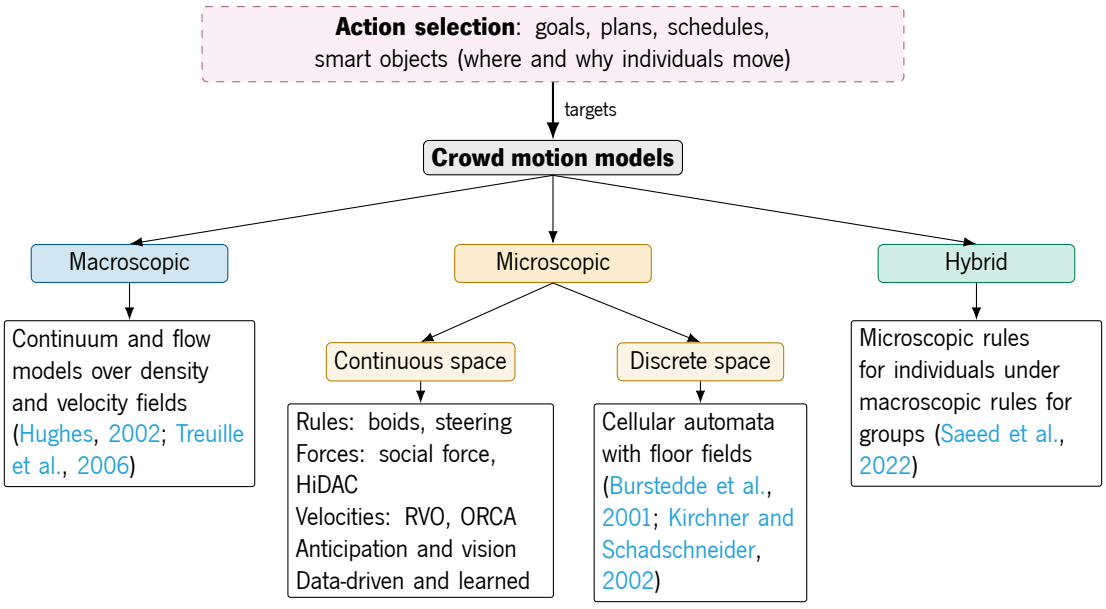


Figure 10: A taxonomy of crowd motion models.

2.4 Purposeful Populations and Shared Resources

Systems that aim to produce believable populations combine decision-making with crowd motion, usually in layers. Hoogendoorn and Bovy (2004) distinguish a strategic level (choosing and scheduling activities), a tactical level (areas and routes) and an operational level (walking), and Musse and Thalmann (2001) and Menge (Curtis et al., 2016) layer control in a similar way. In our system, schedules and stations act at the strategic and tactical levels and the boid-guided paths at the operational level.

Several systems give a population purpose through activities: needs-driven travelers in Pennsylvania Station (Shao and Terzopoulos, 2007), an activity-driven population (Paris and Donikian, 2009), roles and schedules (Pelechano et al., 2008), CAROSA's roles, schedules and parameterized actions (Allbeck, 2010), transit trips generated for a crowd simulator (Ricks et al., 2020) and MATSim's daily activity plans, which are iteratively improved through simulation (Horni et al., 2016), and Table 1 compares them with this thesis. Authored time-of-day schedules are also common in commercial games, as role-playing games such as The Elder Scrolls IV: Oblivion (Bethesda Game Studios, 2006) and V: Skyrim (Bethesda Game Studios, 2011) give their non-player characters daily packages with time windows. Farming games such as Stardew Valley (ConcernedApe, 2016) read the schedules of their characters from data files. Pedestrian simulators for building design, such as MassMotion (Oasys, 2026), Pathfinder (Thunderhead Engineering, 2026), Vadere (Kleinmeier et al., 2019) and JuPedSim (JuPedSim developers, 2026), model services, waiting areas and queues. Neither group combines authored schedules with capacity-limited resources and boids steering inside a general game engine, but both show that schedules and queues on their own are not new, so our contribution lies in their combination and in the timing semantics of Chapter 3.

Saeed et al. (2022) combine microscopic rules for pedestrians, who have attributes such as age, velocity and energy, with macroscopic rules for subgroups that share an intention over several goal points. They sum the steering, obstacle and pedestrian-avoidance forces into one, combine flocking with pathfinding over a navigation mesh (Hart et al., 1968; Snook, 2000) and first test their components in a simple scenario, as the scenario suites of SteerBench (Singh et al., 2009) and RiMEA (RiMEA e.V., 2025) also do. It is the published work closest to our movement design, while Shao and Terzopoulos (2007) and CAROSA (Allbeck, 2010) are closer on timing and resources. As we read it, it addresses motion, with goals and group intentions but no timed schedules or capacity constraints. In their summed forces the flock can pull individuals off their route, while in our hybrid boids the path dominates and steering can only turn the heading by a bounded amount.

Work	Source of purpose	Timing of activities	Shared resources	Local motion
Shao and Terzopoulos (2007)	Internal needs and goals in a cognitive layer	Driven by needs and events	Ticket queues in the station	Reactive behaviors with perception
Paris and Donikian (2009)	Cognitive, goal-oriented activities	Activity-driven	Not a focus	Not a focus
CAROSA (Allbeck, 2010)	Roles and parameterized actions	Authored schedules	Not a focus	Not a focus
Ricks et al. (2020)	Generated trips in a transit facility	Generated arrival and destination times	Not a focus	Separate crowd simulator
MATSim (Horni et al., 2016)	Daily activity plans	Activity times in plans, improved by replanning	Not at pedestrian level	Network-level trips, no steering
Menge (Curtis et al., 2016)	Finite-state goal selection	Not a focus	Not a focus	Replaceable local-avoidance component
Unreal Engine tools (Epic Games, 2026f,e)	Behavior trees or State Tree written by the developer	Developer's own logic	Smart Object slots claimed and released	Navigation mesh paths with avoidance
Saeed et al. (2022)	Group intentions over several goal points	Not a focus	Not a focus	Combined steering forces
This thesis	Authored timed rules in readable files	Rules eligible from a time, deferred while the person is busy	Stations with capacity, FIFO queue and service time	Boids bounded around navigation mesh paths

Table 1: Systems that give simulated populations purpose, where "not a focus" means that the work does not present that aspect.

Once individuals need resources, waiting is part of the outcome. In Kendall's notation, a counter with c places is an M/M/c or, without statistical assumptions, a G/G/c queue (Kendall, 1953). Little's law ($L = \lambda W$) links the mean number in a system to the arrival rate and the mean time spent in it (Little, 1961), and waiting grows sharply as the utilization $\rho = \lambda/(c\mu)$ approaches 1 (Shortle et al., 2018). These results assume steady arrivals, but scheduled demand comes in waves and late starts push later arrivals, so we measure queues rather than calculate them, as discrete-event simulation does (Law, 2015). Commercial pedestrian tools such as the AnyLogic Pedestrian Library combine walking with services, waiting areas and queues (The AnyLogic Company, 2026), but they model a facility, while our setting is a game-engine population with authored schedules. Physical space is a second dimension, as Fruin's levels of service rate areas by space per pedestrian (Fruin, 1971), Weidmann relates walking speed (about 1.34 m/s when free) to density (Weidmann, 1993) and HiDAC models queueing physically (Pelechano et al., 2007). A queue kept as an ordered list ignores these effects, which makes timing analysis much easier but limits the claims that can be made about crowding.

For a purposeful crowd, evacuation and validation remain open. Schadschneider et al. (2009) review evacuation research, whose simulations reproduce clogging and the faster-is-slower effect (Helbing et al.,

2000), and Kuligowski et al. (2010) compare the models in use. In an activity-driven population, individuals are queueing, being served or walking when the alarm sounds, and where they were shapes how they leave. Validation compares a simulation with real behavior, through fundamental diagrams for single files (Seyfried et al., 2005) and for corridors and T-junctions (Zhang et al., 2011), pairwise avoidance against motion capture (Pettré et al., 2009), stop-and-go waves (Lemercier et al., 2012), lane formation and bottleneck oscillations (Helbing et al., 2005), and experiments in which people steer avatars in a virtual environment (Moussaïd et al., 2016) (Fig. 11). SteerBench (Singh et al., 2009), the RiMEA test cases (RiMEA e.V., 2025) and ISO 20414 (International Organization for Standardization, 2020) formalize test scenarios and protocols. Most game-engine crowds, ours included, are not validated to that standard, so we make no claim of behavioral validity, but the thesis provides the instrumentation that validation needs.

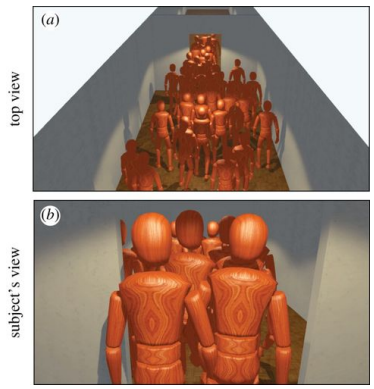


Figure 11: A bottleneck evacuation experiment in a virtual environment, seen (a) from above and (b) by one participant (Figure 1 of Moussaïd et al. (2016), CC BY 4.0, via Wikimedia Commons).

2.5 Discussion

The work reviewed covers each element of our problem but not their combination. Boids and steering give cheap and readable motion (Reynolds, 1999, 2006), crowd models give more realistic motion (van Toll and Pettré, 2021), the engine offers decisions with navigation and object reservation (Epic Games, 2026f,e), timed purpose comes from activity-driven crowds (Shao and Terzopoulos, 2007; Allbeck, 2010; Horni et al., 2016), and queueing theory deals with contention (Little, 1961).

What is missing is their combination in one game-engine system, in which the user specifies timed activities for a reusable boids population and the capacity-limited shared resources, rather than the user, determine when each activity actually takes place. Each system in Table 1 lacks one of these elements, because the research systems have no capacity, queues or engine, the engine tools leave scheduling and queueing to the developer, and Saeed et al. (2022) have goals but no timing or contention. The next chapter turns that gap into a description of the behavior we want and its functional requirements.

Chapter 3

Scheduled Pedestrians and Shared Resources

3.1 Introduction

This chapter sets out what we want to build and what makes it difficult. Section 3.2 describes the behavior we want the people of a simulated place to show, gives a one-page overview of how we produce that behavior (Section 3.2.1) and lists the functional requirements R1 to R7 that arise from it (Section 3.2.2). Section 3.3 then discusses the issues that make these requirements difficult to satisfy in a game engine, which in turn guide the design in Chapter 4 and the assessment in Chapter 6.

3.2 Behavior

We begin with a population of people in the game engine whose movement follows the boids rules, so that they are capable of following a path and of reacting to their neighbors but have no particular reason to go anywhere. A scenario has four components: the *environment* E, the *population* A, the *schedule* S and the *configuration* C. The environment holds the walking surfaces, walls, gates, exits and resources, such as desks, counters and tables, each with its own number of places, the schedule is a set of rules, and the configuration sets the clock speed and the optional behaviors. A rule is of the form $r = (a_r, t_r, \alpha_r, g_r)$, where a_r is the person or group of people that the rule applies to, t_r is the authored time of day, α_r is the action and g_r is the target. The rule becomes *eligible* at $e_r = t_r + o_a$, where $o_a \geq 0$ is an optional per-person offset. The simulation decides when the rule is *dispatched*, at $d_r \geq e_r$, and when the activity *starts*, at $s_r = d_r + q_r + w_r + p_r$, where q_r is the time spent waiting for a free place, w_r the time spent walking and p_r the short positioning at the place before the service starts. The *lateness* of the rule is then $\ell_r = d_r - e_r$, as Fig. 12 shows.

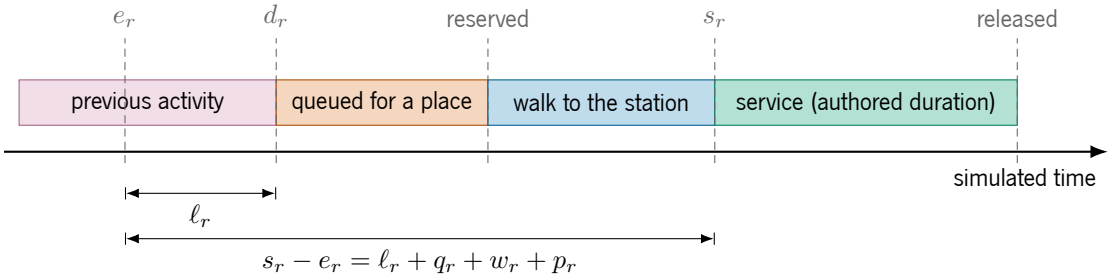


Figure 12: Timeline of one rule that uses a shared resource, where the short positioning p_r is too small to draw (schematic, not measured data).

Given E, A, S and C, the system has to turn the rules into movement and activity such that ℓ_r, q_r and w_r emerge from the people's schedules, their walking, the capacity of the resources and the layout, rather than being explicitly set by the user. It has to do this with one implementation of a person for all scenarios and have sufficient information for the user to see why an activity started when it did. A rule is therefore a request that the simulation may defer, and changing the scenario must mean changing E, S or C, with the person remaining the same.

3.2.1 Overview of the approach

Our approach treats every person in the simulation as someone with a timetable rather than as a particle in a flock. For each person or group of people, a user writes a list of activities, each with the time of day from which it may start and the place where it happens, such as a desk in an office from 08:40 for fifteen minutes or a table in a food court at lunchtime. Nothing in this list says how the person gets there or what happens when the place is taken, because those answers come from the rest of the simulation, whose parts and the signals between them are shown in Fig. 13.

When an activity becomes due and the person is free, the person picks a route to the place and walks it, spreading out from the people around them without being pulled off the route by the crowd. Every shared place, such as a desk, a counter, a table or an exit, has a limited number of places, a first-come queue and a service time. A person asks for a place as soon as the activity starts and either gets one immediately or joins the queue. Once they have the place, the person walks to it, stays for the service time and then frees it, which lets the next person in the queue take the place and lets the person move on to the next step of the activity or to the next item of the timetable. A person who is still busy with something else, such as walking, queueing or being served, keeps the next item waiting until they are free, while an item that has become far too late is dropped.

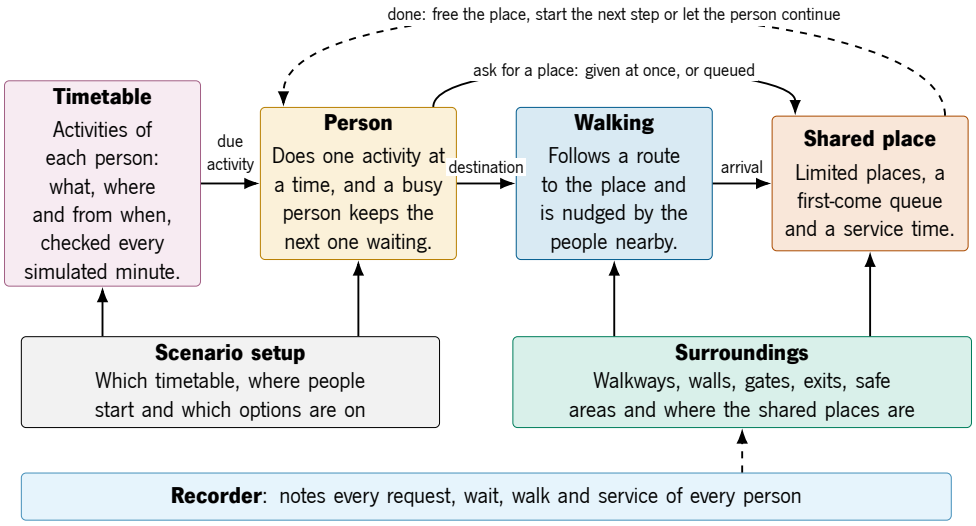


Figure 13: Overview of the approach, where the dashed arrow at the top is the signal that lets a person continue once an activity is done.

The surroundings shape all of this without appearing in the timetable, as walkways set the preferred routes, walls and closed gates block them, exits and safe areas take over when an alarm sounds, and the number and position of the shared places decide where queues form. A separate setup for each scenario only says which timetable to use and where people start. A recorder notes every request, wait, walk and service, so that the gap between the written time of an activity and the time it really started can be explained afterwards. The written time is therefore a request rather than an order, and the actual start depends on what the person is still doing, how far they have to walk and whether the place is free, and that gap is what the evaluation in Chapter 5 measures.

3.2.2 Functional requirements

Table 2 lists the functional requirements that this behavior implies. None of them require realistic behavior, collision-free motion or the ability to predict a real facility.

3.3 Issues

3.3.1 Purpose versus motion

Boids react to the neighbors in their field of view (Reynolds, 1987), while purpose is global, individual, and changes throughout the day. If goal forces are added to the steering sum, the flock can pull people away from their goals through weights that mean nothing to the user, and hard-coded purpose turns every

activity into a separate piece of code. With the separation of action selection and steering (Reynolds, 1999), purpose should be in the form of readable rules external to the person (using a small vocabulary, **R1**). Reaching a target around walls and roads requires a path over a navigation mesh (Snook, 2000; Mononen, 2009), but a person who simply follows a path loses all the local interactions of the boids. The path tells the person where to go, and the boids tell them how to get through the others, while the environment defines which surfaces are preferred and which routes are available (**R5**).

ID	Name	Requirement
R1	Purpose as authored data	Which people do what, when and where is stated in readable files, such as XML schedules, apart from the simulation code, with a small fixed set of actions, and changing activities needs no recompilation.
R2	Timing under contention	An authored time is the earliest start. Eligible, deferred, dispatched, used-up and dropped rules are defined, no authored minute is skipped, realized times result from the simulation, and durations keep their meaning in every run mode.
R3	Shared capacity-limited resources	Resources have places, a waiting discipline and a service time, people reserve or queue, places are released on completion or interruption, and multi-stage activities advance on completion.
R4	Reuse across scenarios	One implementation of a person, one movement layer and one resource model serve different maps and activity patterns, and scenario-specific code is small, isolated and identifiable.
R5	Environmental constraints	Surfaces, walls, gates, exits and the number of resources shape movement and timing without being restated in rules, and people follow paths while boids shape local motion.
R6	Observability and measurement	Runs record authored, eligible and realized times, resource events, frame timings and the state of every person in files analysable outside the engine, with documented measures.
R7	Interactive performance	Scenarios run at 30 frames per second with their authored populations, a faster non-visual mode exists, and cost is measurable as the population grows.

Table 2: Functional requirements that follow from the behavior we want.

3.3.2 Timing under contention

At 08:40, a person could still be walking from home or could still be waiting, as someone else has the desk. Executing the rule anyway would break the routine, executing the rules strictly in order could delay the shift for the whole day, and waiting indefinitely would let the morning rules run into the afternoon. The system has to define when a rule is eligible, when it is deferred, when it is used up, and when it is dropped. No authored minute may be skipped, even when one frame covers several simulated minutes, and the durations must have the same meaning in every run mode (**R2**).

3.3.3 Shared resources and queues

Desks, counters, tables and exits are all servers with a capacity, an arrival process, driven here by schedules and walking, and a service time (Kendall, 1953) (Fig. 14). The design must decide whether a place is booked on request or on arrival, where the person queues and for which station, and what releases the place. It must also handle a two-stage activity, such as ordering and then eating, so that a person does not hold a table while still queueing at the counter. Smart objects hold the information needed to use them (Kallmann and Thalmann, 1999), and Unreal Engine's Smart Objects let people claim a slot (Epic Games, 2026e). We also need capacity, a queue and a timed service linked to the schedule, with a lifecycle detailed enough to measure the time spent waiting (**R3**).

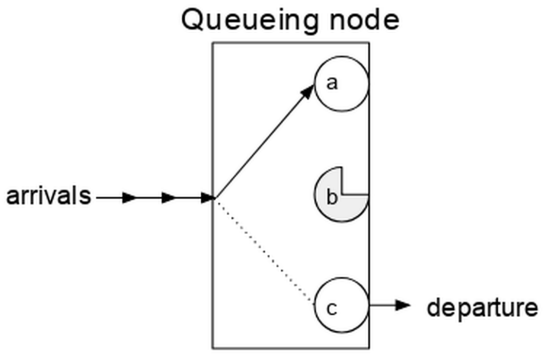


Figure 14: A queueing node, where arrivals wait for one of several servers and leave after their service (diagram by Dt-rush-8, CC BY-SA 4.0, via Wikimedia Commons).

3.3.4 Reuse across scenarios

The same implementation of a person, movement layer and resource model should serve a daily urban routine, a shopping center, an evacuation and a directed-flow test, with the variations expressed in schedules, maps and configuration. Full reuse is difficult, because an evacuation needs exit choice and a routine needs a home, so scenario-specific code should be minimal and easy to identify (**R4**), and the environment should absorb variations such as an extra table or a gate opened by a button (**R5**).

3.3.5 Measurement and performance

A person who stands still could be queued, deferred, blocked or stuck, so watching a run does not reveal their state. The system must therefore record rule times, station events, trips and frame times, document what each measure means and reproduce the analysis from the files (**R6**). It also shares each 33 ms frame with rendering and animation, and naive boids compare every pair of people (Section 2.1.2), so the scenarios must run interactively at a cost that can be measured (**R7**).

Part II

Core of the Dissertation

Chapter 4

A Rule-Authoring Framework for Pedestrian Activity

4.1 Introduction

The framework separates four concerns that a crowd demonstration usually mixes in one controller, following the overview of Section 3.2.1. These are *what* the person should do and *when* (the schedules), *how* they get there (path-guided hybrid boids), *whether* the activity can take place at that moment (capacity-limited stations) and *what the place actually allows* (the environment). Table 3 shows how the mechanisms of the framework map onto the functional requirements of Chapter 3.

Requirement	**Where it is addressed**
R1 Purpose as authored data	XML schedules with timed, per-person and cohort rules (Section 4.3.2), six actions (Section 4.3.3).
R2 Timing under contention	Eligibility, deferral, consumption at dispatch, stale skip (Section 4.3.2), minute-by-minute clock (Section 4.3.7).
R3 Shared capacity-limited resources	Stations, areas, super stations and pipelines with reservation, FIFO queue and timed service (Section 4.3.5).
R4 Reuse across scenarios	One class for every person, one parser and one station model, scenario managers that prepare each map (Sections 4.3.1 and 4.3.6).
R5 Environment constraints	Sidewalk filter, gates and buttons, exits, safe zones (Section 4.3.6), `Interact` (Section 4.3.3).
R6 Observability and measurement	Summary exports and a validation panel, and for the evaluation a run recorder on the test branch (Section 4.3.8).
R7 Interactive performance	Visual and Accurate modes (Section 4.3.7) and bounded boids blending, measured by a stress test on the test branch.

Table 3: Requirements and where the framework addresses them.

Section 4.2 describes the first approach, built before the dissertation, and the two changes that arose from it. Section 4.3 then describes the current pedestrian movement simulator, from its schedules, actions, movement, stations and environment to its clock and run modes, configuration and instrumentation, and Section 4.4 summarizes the chapter.

4.2 First Approach

The work started from a prototype built during the pre-dissertation phase, in which a flock of boids moved through a city map and external rules tried to give it purpose. Its rules were organized around categories of places rather than around people. An XML file gave each object of the level a role, such as home, workplace, food place or walkable surface, with fixed active periods such as a morning shift, and during an active period every boid went to a random place of the matching category. Purpose therefore belonged to the period and the type of place rather than to a particular person, and nothing represented a resource that people could share or wait for. Since the boids flocked as one group and steered straight to their destination, a boid that left its group was either held back by its neighbors or dragged them along.

The current system changes both aspects, as rules now name the person, or the group of people, they belong to, give the time of day from which an activity may start and address places through tags and stations with capacities, queues and service times (Sections 4.3.2 and 4.3.5). Movement became individual, as each person follows their own navigation path and the boids rules only bend their heading by a bounded angle, so that the flock spreads people around one another without deciding where they go (Section 4.3.4).

4.3 Pedestrian Movement Simulator Architecture

The current system replaced the prototype step by step, starting with a schedule manager that gives named people timed tasks. Navigation paths, areas and stations, the scenarios, the hybrid blend, super groups and super stations followed, and finally the simulated clock, the run modes and the action objects, while the division of responsibilities of Fig. 13 stayed the same throughout.

4.3.1 Architecture and module layout

All code lives in one runtime module, `Thesis`, which uses the engine's `XmlParser`, `AIModule` and `NavigationSystem` modules. Figure 15 groups the classes by layer, with Unreal's prefixes for actors (`A`), objects and subsystems (`U`), structures (`F`) and interfaces (`I`).

`USimulationFeatureRegistry` loads `SimulationConfig.xml` on every Play, holds the feature flags and settings and answers any other class that asks whether a feature is on. The `AScheduleManager`, one per level, owns the clock, loads the schedule, maintains a registry of people by identifier and evaluates the due rules once per simulated minute. `ABoidsAB` is the only class

that represents a person. As an `ACharacter`, it leaves walking, gravity and collision with the world to the engine's character movement component. It has a 300 cm perception sphere for its boids neighbors and an `AgentId`, usually `Agent_N`, that links it to the schedule, and it carries optional components for needs, the priority selector, failure handling, leadership, the police-and-robbers role and greetings, so that one Blueprint of a person serves every scenario. Stations, areas and environment actors are placed in the level and found by tag, spawners create the people and an identifier manager names them in spawn order. Scenario managers prepare a run rather than drive it: at its start, they pick the schedule file, assign homes or entrances and place the people there, set station capacities and start the analytics sessions. Later, the emergency manager raises the alarm and runs the countdown, while each person chooses their own exit and walks there.

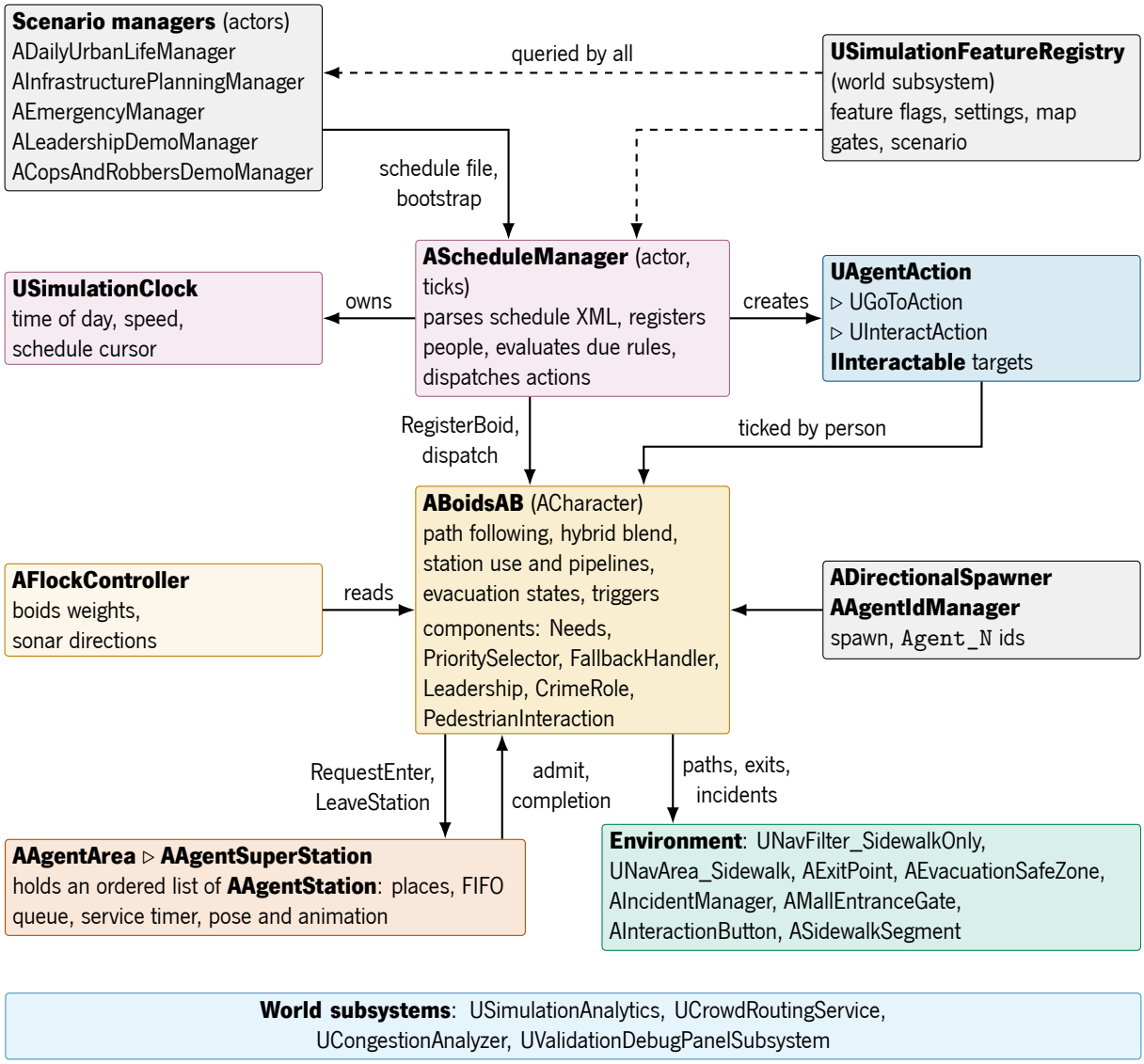


Figure 15: Classes of the second implementation grouped by layer, where solid arrows are calls or ownership and dashed arrows are queries of the configuration registry.

In each frame, the schedule manager advances the clock and processes every simulated minute between its cursor and the clock. Each person then updates their current action, their station pose and their needs, lets the priority selector choose the controlling layer and moves along their path. The rest is event-driven, as engine timers in real seconds end station services and timed idles, process exit queues and run the evacuation countdown. People retry registration every 0.2 s until the manager accepts them, so actor order does not matter.

4.3.2 Schedules and the rule language

A schedule is an XML file in `Project/Schedules/` that tells named people what to do and from when. The schedule manager of each level names its file, and the Daily Urban Life and infrastructure managers can replace it with a file named in the configuration. A schedule has a root `<schedule>` with `<agent id="Agent_N">` and `<superGroup>` blocks, a person has `<rule time="HH:MM">` elements, and each rule may have an `<if condition="..."/>`, a `<then action="A" .../>` and an `<else>` alternative. A `UseStation` action may have `<stage>` children, and a super group has a `<members>` list and its own rules, and Table 4 lists every element with its attributes. Element names are case-insensitive, but attribute names must be spelled exactly, and a missing or malformed file is logged and leaves the people without rules.

Element	Attributes (default)	Meaning
`schedule`	none	Root of the file, holding `agent` and `superGroup` blocks.
`agent`	`id` (required)	The rules of one person, whose `id` must equal the runtime identifier `Agent_N`.
`superGroup`	`id`	A cohort whose `members` list names people with `<agent ref="Agent_N"/>`, each of whom receives a copy of the group's rules.
`rule`	`time` (00:00), `prob` (1.0)	The time of day from which the rule may run, where malformed text gives 00:00 and out-of-range values are clamped, and a probability rolled at every attempt, where a failed roll uses up the rule.
`if`	`condition` (none)	`emergency == true|false` or `location == Home|Work|Food`, which selects `then` or `else`.
`then`, `else`	`action`, `area` or `superStation` or `target`, `stationType`, `duration` (-1), `id`	One of the six actions of Table 6, its target tag (the first of the three that is given), a station identifier or tag inside the area, a duration in simulated minutes (≤ 0 gives an indefinite idle or the station's default of 5) and, for `Trigger`, the trigger name.
`stage`	`area` or `superStation`, `stationType`, `duration`	One step of a `UseStation` pipeline, with the same meaning as above.

Table 4: Elements of the schedule language with their attributes and defaults.

Listing 4.1 is the full schedule of one office worker in the Daily Urban Life scenario. The worker uses a bed in the `Sleep` area, walks to the office area `Work0`, sits at desk `A0` for 15 simulated minutes, takes a break, works a second block, walks home and goes back to bed. The gaps between the rules allow for walking, and the evaluation measures whether they are long enough, since a rule that comes due while the person is still walking is deferred. The file holds 50 people with 304 rules, and people 25 to 49 repeat the pattern of people 0 to 24 fifteen minutes later on the same desks, so a late first user makes the second user queue. The file creates this contention without any scripting, because it only states who wants which desk and from when.

```
<agent id="Agent_0">
 <rule time="08:05"><then action="UseStation" area="Sleep" stationType="Bed1" duration="20"/></rule>
 <rule time="08:22"><then action="MoveTo" area="Work0"/></rule>
 <rule time="08:40"><then action="UseStation" area="Work0" stationType="A0" duration="15"/></rule>
 <rule time="09:00"><then action="Trigger" id="Break"/></rule>
 <rule time="09:13"><then action="UseStation" area="Work0" stationType="A0" duration="15"/></rule>
 <rule time="09:43"><then action="MoveTo" area="Home1"/></rule>
 <rule time="09:55"><then action="UseStation" area="Sleep" stationType="Bed1" duration="20"/></rule>
</agent>
```

Listing 4.1: Schedule of `Agent_0` in `DailyUrbanLifeSchedule.xml`, complete.

Listing 4.2 is one shopper in Mall A, whose lunch rule in the super station `FoodCourt` has two stages, 10 minutes at a counter and then 14 minutes at a table. The table is sought only when the counter service has finished, so a queue at the counter delays the search for a table. The two `<stage>` elements are children of the rule's single `<then>`, which the closing `</then>` ends, and the rule needs no `<else>` because a rule without an `<if>` always runs its `<then>`. A `<superGroup>` is written in the same way, with a `<members>` list before the rules, and each corridor map has one or two of them, one per cohort.

```
<agent id="Agent_0">
 <rule time="08:02"><then action="Idle" duration="8"/></rule>
 <rule time="08:12"><then action="MoveTo" area="ShopA"/></rule>
 <rule time="08:22"><then action="UseStation" area="ShopA" stationType="ShopA_Desk" duration="10"/></rule>
 <rule time="08:36"><then action="MoveTo" area="FoodCourt"/></rule>
 <rule time="08:46">
  <then action="UseStation">
   <stage superStation="FoodCourt" stationType="FoodCounter" duration="10"/>
   <stage superStation="FoodCourt" stationType="FoodTable" duration="14"/>
  </then>
 </rule>
 <rule time="09:16"><then action="MoveTo" area="Exit1"/></rule>
</agent>
```

Listing 4.2: Schedule of `Agent_0` in `InfrastructurePlanningSchedule_MallA.xml`, complete.

A rule is *due* at the processed minute M of day D if it has not been consumed on day D and its effective minute is not later than M:

$$\boxed{\text{due}(r, M, D) \iff \text{LastExecutedDay}(r) \neq D \ \wedge \ (t_r + o_a) \bmod 1440 \leq M} \tag{4.1}$$

where t_r is the authored time in minutes since midnight and o_a is the person's schedule offset. For each person, the manager visits the due rules in time order and makes one attempt per rule, which

either *dispatches* it (its action starts), *defers* it (it is tried again at the next minute) or *consumes* it without dispatch. Before any other check, a rule of a person who is present and not busy is consumed as stale when more than 45 minutes have passed since its effective minute, while the rules of a person who has not spawned yet are kept until that person appears. A busy person is never stale-skipped, so an hour in a queue does not cost them every rule that came due meanwhile, and the window, set by hand, stops a long morning delay from releasing morning activities into the afternoon. The probability is then rolled at every attempt, so a rule deferred k times is dispatched with probability p^{k+1} rather than p, although no shipped schedule sets `prob`, and a false condition runs the `<else>` action if there is one and otherwise consumes the rule.

Table 5 lists the checks that defer a rule, in the order they are applied. The exemptions are as important as the checks themselves, because `StartEvacuation` must not wait for anyone, a `Break` must interrupt a desk, and in Daily Urban Life a commute home must interrupt whatever the person is doing.

	Reason	**Exemptions**
a	Person not registered with the schedule manager	none
b	Person in an evacuation state (the condition is evaluated after this check)	none
c	An incident or a global evacuation is active in the world	`StartEvacuation`
d	Person at a station, queued for one, or in a pipeline	`StartEvacuation`, `Trigger Break/Pause`, in Daily Urban Life a `MoveTo` to a `Home` tag
e	Person's previous action still running	`StartEvacuation`
f	Critical hunger or energy, an active crime response, or following a leader	in Daily Urban Life, `MoveTo/Interact` to a `Home` tag

Table 5: Reasons for deferring a due rule, in the order they are checked.

Deferral turns walking and queueing time into lateness, since a rule that comes due while the person is still walking waits until the person arrives. It also lets rules run out of the order in which the user wrote them, as when a `Home` commute, which is exempt from the station check, overtakes a desk rule that is waiting for its desk. A rule is consumed when its action is dispatched, not when the activity ends, so the scheduler never needs to know how an activity ends. A user expresses "do B after A has finished" either as stages of one `UseStation` rule or through deferral, which holds the next rule while the person is busy. A `<superGroup>` gives each member its own copy of its rules, which are evaluated, deferred and consumed one person at a time, so a cohort saves the user from writing every rule but does not model a collective decision. In Daily Urban Life, each person's rules can be shifted by a random offset of up to 8

minutes, so that people sharing a pattern do not all leave in the same minute.

4.3.3 Actions

The action vocabulary is kept small on purpose (Table 6), because a larger vocabulary would make schedules harder to read and let scenario logic creep back into the parser. `UAgentAction` has four states (not started, running, completed and failed), and a person owns at most one action at a time, so a new action cancels its predecessor. The action object does not move the person but calls the person's normal movement functions and watches the person's state, so movement code stays in one place and deferral check (e) only asks whether the action is still running. A `GoTo` completes when the person reports arrival or has stopped within 180 cm of the target, and fails when the person is still moving without a path after 0.5 real seconds, or after 180 real seconds unless the person is then within 180 cm. Every target is a single point, so all people bound for `Exit1` end at the same point, and any stop of movement sets the arrival flags, which is why the run recorder of the evaluation also logs the remaining distance. `Interact` needs a target that implements `IInteractable`, and its only implementation is the Mall B entrance button, which opens a linked gate and so changes the navigation mesh for later path queries.

Action	Object	What happens
`MoveTo` / `GoTo`	yes	Walk to the first actor carrying the tag. Completes on arrival within 180 cm and fails on path failure or after 180 real seconds (90 simulated minutes in Accurate mode).
`Interact`	yes	Walk to the target's interaction point, then call `Interact` on it if it still accepts.
`Idle`	no	Stop and stay idle for `duration` simulated minutes (indefinitely if ≤ 0).
`UseStation`	no	Start a station pipeline of one or more stages (Section 4.3.5).
`Trigger`	no	Raise one of the named triggers `Break`, `Pause`, `Flee`, `Greet`.
`StartEvacuation`	no	Start the global evacuation through the emergency manager.

Table 6: The six schedule actions, where "Object" marks the actions that create a `UAgentAction`.

`Idle` stops the person and starts a timer, and the mall schedules open with idles of 2 to 10 minutes so that shoppers do not all enter at once. `Break` and `Pause` release the person from their station, and in Daily Urban Life a `Break` sends the person to the plaza. `UseStation` builds the stage list and starts the station pipeline, and the rule is consumed even if no stage can be built. `StartEvacuation` calls the emergency manager, which starts a fire incident of global radius, alarms every person and starts the countdown. The call is global and idempotent, so the evacuation schedule is a single rule for `Agent_0`, after which check (c) holds every other rule of every person.

4.3.4 Movement: capsules, probes and the hybrid blend

Each person's capsule is moved by the engine's `CharacterMovementComponent`, which applies gravity and slides it along blocking geometry, and the boids rules see the people who overlap a 300 cm sphere around the person. There is no grid and no cap on the number of neighbors, so the cost per person grows with local density (Reynolds, 2006). None of this is collision avoidance in the pedestrian sense, where each person would choose a velocity that keeps them clear of their neighbors over a time horizon (van den Berg et al., 2008, 2011; Karamouzas et al., 2014), so overlaps are possible and capsules deal with contact afterwards. A forward sonar ray of 300 cm turns the person toward the first free one of 100 precomputed directions when the ray hits an obstacle (Reynolds, 1999). In Visual and Accurate modes (Section 4.3.7), two side rays of 95 cm and a forward ray of 110 cm also remove any component of movement toward a wall and slide the person along walls they would hit. A person pushed more than 50 cm off the navigation mesh is projected back onto it at most once per second. The fallback handler retries failed path queries, recomputes the path of a person who covers less than 15 cm in 6 s, and lets a fleeing person choose another exit when more than three paths to their queue slot have failed.

In the hybrid blend, the path decides the direction of travel and the flock can only turn it sideways by a bounded angle. The progress direction $\hat{\mathbf{p}}$ points to the point 140 cm ahead of the person's closest point on their path segment, never backwards along it. The flock force is flattened, normalized and scaled by $w_m = 0.35$, its component along $\hat{\mathbf{p}}$ is discarded, the lateral remainder $\mathbf{l}$ is capped at $\tan 18^\circ \approx 0.325$, and the new heading is the unit vector

$$\mathbf{h} = \frac{\hat{\mathbf{p}} + \mathbf{l}}{\|\hat{\mathbf{p}} + \mathbf{l}\|} \tag{4.2}$$

shown in Fig. 16. The forward component is always the full $\hat{\mathbf{p}}$, so the flock can never stop or reverse progress, and the wall probes then refine the direction within the same cone. The blend is switched off in Fast mode, in every evacuation state and while a person is admitted to or queued for a station, so that a worker going to a reserved desk reaches it, and in those cases the path alone decides the direction.

The priority selector decides each frame which layer controls the person (Table 7). Higher layers move the person through the layer's own overrides and defer the person's due rules until the layer releases them, and in our tests crime and leadership were off and needs were frozen, so only the emergency, schedule and idle layers were active. The outcome is two regimes: *free flocking*, where the boids rules alone set the velocity, and *directed travel*, where a rule, need, leader or evacuation provides a target, the path sets the direction and the flock only nudges it. We preferred this to the usual weighted sum of seek and flock forces (Reynolds, 1999), in which a dense group can stall a person or carry them past their turn and the

right weights depend on density. The downside is that the flock has little influence during travel, which is when crowding occurs.

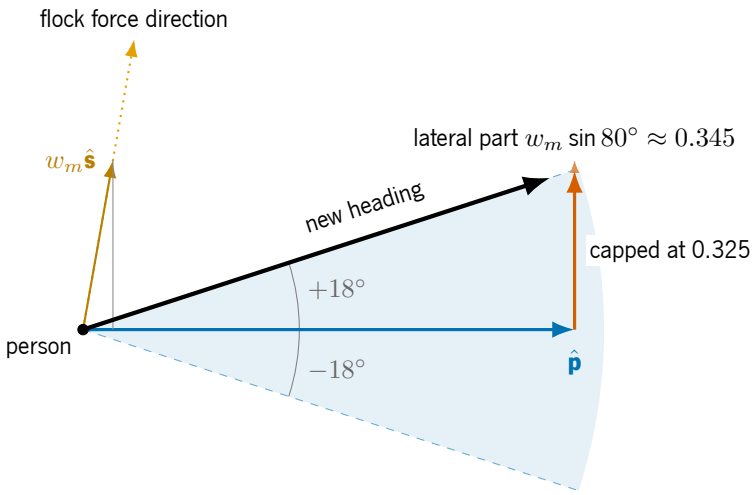


Figure 16: Geometry of the hybrid blend, where the heading stays inside the shaded cone around the path. The flock force is drawn at 80° to the path as one example, so its lateral part is $w_m \sin 80° \approx 0.345$ and the cap at $\tan 18° \approx 0.325$ shortens it.

Layer	Applies when	Movement and effect on the schedule
PanicEmergency	fleeing, exiting, awaiting reaction, moving to a safe zone, evacuated or casualty	evacuation movement, all rules deferred
CriticalNeeds	hunger ≥ 0.65 or energy ≤ 0.35	to the nearest food point or home, leaving any station, rules deferred
CrimeResponse	crime feature on and the role wants to pursue, flee or avoid	run to or from the other party, rules deferred
Leadership	leadership feature on and actively following	walk behind the leader, rules deferred if configured
ScheduledTasks	moving or holding a target station	schedule action with the blend
IdleFlock	none of the above	free flocking, or a slow mill near home in Daily Urban Life

Table 7: Priority layers of the person, highest first.

4.3.5 Stations, areas, super stations and pipelines

A rule that sends a person to the office has no way of knowing whether a desk will be free, and stations are what keep track of that. They build on smart objects, which hold the logic of their use (Kallmann and Thalmann, 1999; Forbus and Wright, 2001), and on Unreal Engine's Smart Objects, whose slots people claim and release (Epic Games, 2026e), and they add several places, a first-come, first-served queue and a timed service authored in the schedule. In queueing terms, a station is a multi-server FIFO queue (Kendall, 1953; Shortle et al., 2018) with two differences from the standard model, as places are booked before the customer arrives and the queue takes up no space. An `AAgentStation` is one resource,

such as a desk, counter, table or bed, with its places, its admitted occupants, including those still walking, and a queue. An `AAgentArea` groups stations under an actor tag such as `Work0` in an ordered array, and an `AAgentSuperStation` is an area of a few interchangeable stations, such as the counters and tables of a food court. A pipeline is the person's list of stages with the index of the current one, and Fig. 17 shows one stage.

A stage is resolved to a station only when it starts, so a diner does not hold a table while still queueing at the counter (Listing 4.2). `RequestEnter` admits the person if a place is free and otherwise queues them. Reserving at dispatch prevents two people from walking to the last desk and finding the conflict on arrival, although the desk then sits idle while its user walks and the first person to ask wins even if another is nearer. A queued person becomes idle where they stand and does not walk to the station or line up. An admitted person walks to the station and arrives within 180 cm horizontally and 220 cm vertically of it, then snaps or interpolates to the station's interaction point, so that a diner sits at the table rather than beside it, and their service timer and animation start. The service lasts the stage's `duration` or, if none is given, the station's default of 5 simulated minutes, converted to real seconds for the run mode (Section 4.3.7). When it expires, the place is released, the next stage is resolved from where the person stands, and the station admits the head of its queue on its next tick. `LeaveStation` frees a place or a queue entry when a `Break`, `Pause`, home commute, evacuation, need or queue time-out interrupts the person, and while a person is admitted, queued or in a pipeline, their other rules are deferred, which links station waiting to schedule lateness.

A stage resolves to the first open area with the stage's tag and, within it, to the first station in array order whose type matches and that has a free place, or otherwise to the first matching station. A misspelled `stationType` therefore silently falls back to any station in the area. This policy had two effects in the malls. First, a queued person stays bound to the station chosen at that moment and does not move to an equivalent one that frees earlier. Second, when every station is busy, all new arrivals join the queue of the *first* matching station, so the array order becomes a modeling decision. A queued person gives up after 50 engine seconds and goes home, except in the infrastructure scenario or when `enableInfrastructureAnalysis` is on, where they wait indefinitely. There is no time-out between admission and arrival, so a place stays reserved until its owner arrives or leaves.

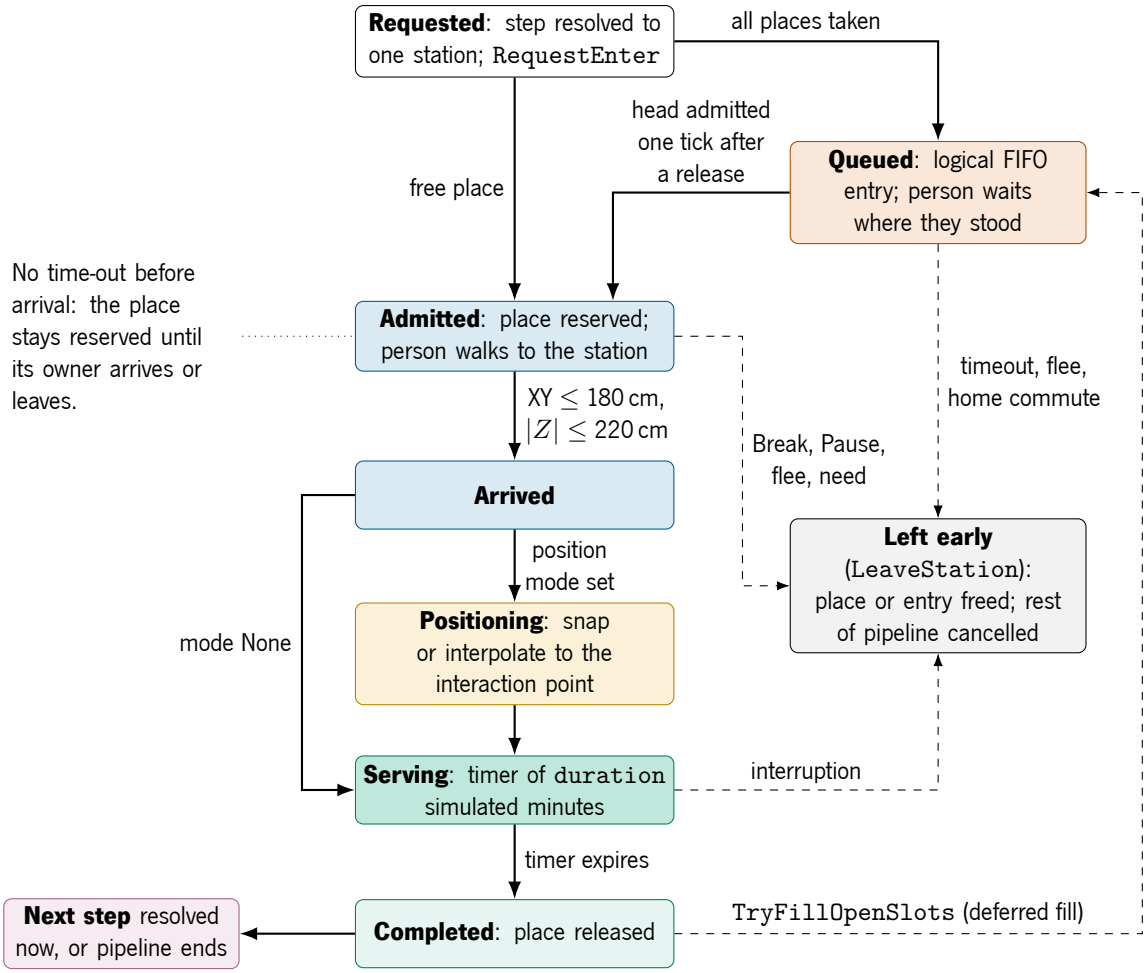


Figure 17: Lifecycle of one station step, where dashed arrows are interruptions.

4.3.6 Environment and scenario managers

Paths come from the engine's Recast navigation mesh and from Detour's A* search (Snook, 2000; Mononen, 2009; Hart et al., 1968; Epic Games, 2026d), queried on the game thread with one filter, `UNavFilter_SidewalkOnly`, which gives the default area a very high travel cost and sidewalks, painted with navigation modifier volumes, a cost of 1. Roads are therefore penalized rather than forbidden, and walls need no special actor, because they block capsules, cut the mesh and are seen by the probes. Mall B's second entrance has a gate that blocks movement and marks its doorway as non-navigable while closed, and when a person presses the gate's button, the gate opens and the mesh is rebuilt locally, so only later path queries use the doorway. An `AExitPoint` lays out queue slots 120 cm apart behind the exit and grants each slot only if it is on the sidewalk mesh and reachable. It serves one person at a time, as the front person must reach their slot and the exit then waits 1.5 real seconds before releasing them. People score exits by distance and queue length, re-evaluate them periodically and may switch a bounded number of times. Once released, a person walks to the nearest safe zone and counts as safe

within 200 cm of it, and since all people go to the same safe zone, early arrivals at its rim can block later ones. When the countdown of the emergency manager ends, every person not yet safe is marked as a casualty.

Each family of maps has a manager actor that runs once the spawners have created the people. The `ADailyUrbanLifeManager` gives each `Agent_N` a home and a work tag from its number and one of the shared food points, draws its schedule offset and a walking speed between 170 and 230 cm/s, and scales its needs rates by a configured factor. The `AInfrastructurePlanningManager` picks the mall layout and schedule, sets the capacity of every station that carries a configured actor tag, places people at the entrances and ends the run after a set number of simulated minutes. Some features are switched on or off from the map name, which lets one configuration file serve every map during development, at the cost of a map changing its behavior when it is renamed.

4.3.7 Simulation clock and run modes

`USimulationClock` holds the simulated seconds since midnight of day zero, a speed in simulated seconds per engine second and a start time. The start time is set in `<simulationClock>` with a default of 08:00, except on the directed-flow test maps, whose schedules count from midnight and whose clock therefore always starts at 00:00. Schedule times are written as `HH:MM`, and the default speed of 60 makes one real second one simulated minute, the pace for which every shipped schedule was written. Rules are tested against a *schedule cursor*, which the manager advances one minute at a time while it is behind the clock, up to 120 minutes per frame so that a long hitch does not freeze the game. No authored minute is therefore ever skipped, which could happen in the earlier version that evaluated one minute per frame.

`<simulationMode mode="Visual|Fast|AccurateTime"/>` selects the run mode, where Visual mode, the default, is what the viewer sees, and Fast mode runs schedules at a higher clock speed for quick checks, although no result of Chapter 5 uses it. In Visual mode, walking is not scaled to the clock, so a person moving at about 2 m per real second takes about 50 real seconds for a 100 m walk, which the clock counts as 50 simulated minutes. Relative to the time spent at a station, every walk therefore lasts 60 times longer than the people's own speed implies.

Accurate mode removes this distortion and is the mode of every result in Chapter 5. It locks walking to the clock with one scale, `accurateTime.simulationSpeed` (30 by default), so the clock advances by the real frame time multiplied by 30 and every walking and fleeing speed is multiplied by 30. Acceleration is raised as well, so that a person reaches their speed within one movement sub-

step. At 30 fps, a frame is one simulated second, and at any frame rate the clock and movement never drift, so a 100 m walk at 2 m/s takes 50 simulated seconds. Engine timers still count real seconds, so `ScaleAuthoredDurationToRealSeconds` converts every authored time written for the Visual pace into the authored value $\times 60/30$ real seconds, which the clock counts as the same number of simulated minutes as in Visual mode. This covers station and `Idle` durations, interaction pauses, evacuation reaction delays and the evacuation countdown, so a 10-minute service lasts 600 simulated seconds and the 120-second countdown lasts 120 simulated minutes. Three constants do not go through this conversion and continue to count real seconds: the GoTo time-out (180 real seconds, which the clock counts as 90 simulated minutes), the queue time-out (50 real seconds, or 25 simulated minutes) and the exit processing (1.5 real seconds, or 45 simulated seconds).

A step of one simulated second moves a person about 2 m, and the test branch used for the evaluation adds four adjustments for such steps (Section 5.2.1). A run in Accurate mode ends at its duration cap, or earlier when no person is busy and every rule has been consumed, and since people cross a room in a few frames, Visual mode remains the mode for watching and debugging. The schedule manager applies the seed at `BeginPlay`, and all randomness shares the engine's global stream. With variable frame times, a seed therefore does not make a run reproducible, while with a fixed time step it reproduced the Mall A runs exactly but not the corridor or Daily Urban Life runs (Appendix A.1).

4.3.8 Configuration and instrumentation

Settings that belong in neither the map nor the schedule are kept in `SimulationConfig.xml`, so that a reader can see in one file which settings a run used, in keeping with the **ODD** protocol (Grimm et al., 2020). The registry resets to its code defaults, loads the file, in which each section only overwrites the options it contains, and applies the map-name gates, at world creation and again at every Play. Parsing is lenient, as tags are matched case-insensitively, malformed numbers become 0 and unknown tags are ignored, so a misspelled option silently keeps its default. The commented file and the code defaults are in the public repository (Armada, 2026). For the experiments of Chapter 5, a runner wrote one configuration per run and copied generated schedules into `Saved/Schedules/`, which the game reads when no project file of that name exists, and Table 9 lists every setting that the runs changed.

The built-in exports of the framework write summary files at the end of a run, and the evaluation uses only the evacuation summary. Its other numbers come from a run recorder (`URunRecorderSubsystem`) written for the evaluation on a test branch, which logs events rather than summaries when the command line contains `-RunRecord=<Label>` and writes the files of Table 8, with simulated time in minutes since

midnight of day 0. A trip's `arrive` event means that the GoTo completed, which also happens when any stop of movement is in play, so `remaining_cm` shows whether the target was actually reached. With `-RecordPositions=<s>`, it also logs every person's position, velocity and capsule radius for the blend evaluation.

The stress test (`UStressTestSubsystem`, enabled by `-StressAgents=N`) clones the existing people until N are present, waits for a warm-up, measures frame, game-thread, render-thread and **GPU** times and peak memory, writes a summary file and quits. With `-StressHideMeshes`, it hides the characters and stops their animation and hair, so that the run measures the framework without them, and `-NoBlend` turns the blend off for its evaluation. The recorder and the stress test are both on a test branch of the public repository (Armada, 2026), with the command lines of Appendix B.

4.3.9 Extending the framework

A new scenario that uses the existing actions needs a map with one `AFlockController`, one `AScheduleManager`, people or spawners, stations grouped into tagged areas, a navigation mesh with painted sidewalks, destination actors tagged as the schedule names them and a schedule file, while a new type of station needs no code at all. Code is needed only when the vocabulary cannot express something. A new action needs a value in the action enumeration, a parser case, an execution branch and, if it must be followed until it ends, a `UAgentAction` subclass, which is how `Interact` was added together with its button and gate. A new use case also needs a manager actor that owns the run, and these are the scenario-specific pieces of Chapter 6. The main branch has one automated test, which checks that Accurate mode scales walking distance and authored durations consistently with the clock.

File	Columns	Meaning
`rules.csv`	`agent, authored_min, eligible_min, dispatch_min, day, minutes_late, deferrals, action, outcome`	one row per consumed rule, outcome `dispatched`, `stale_skip` or `prob_skip`
`stations.csv`	`real_s, sim_min, station, agent, event, queue_len, occupants`	`admit`, `queue`, `arrive`, `start`, `complete`, `leave`, `leave_queue`, `admit_from_queue`
`trips.csv`	`real_s, sim_min, agent, target, event, detail, remaining_cm`	GoTo `start`, `arrive`, `fail` with reason and distance left
`timeline.csv`	`real_s, sim_min, frames, frame_ms_mean, frame_ms_max, game_thread_ms, gpu_ms, agents, moving, idle, queued, at_station, fleeing, safe, casualty`	once per second: frame timing and people by state

Table 8: Files written by the run recorder.

Setting	Experiment value and reason
`enableLeadership`, `enableCopsAndRobbers`	false in every test, so only schedules, stations, movement and environment act
`enableInfrastructure Analysis`	on in every test and on every map: idle people stop instead of flocking, a queued person waits without time-out, and a failed path stops the person with their reservation
`simulationMode`, clock start	`AccurateTime` with `simulationSpeed` 30 for every behavioral run, from 07:30 (before the first rule, so the mesh is ready) or, in the evacuation, from the default of 08:00, and Visual mode for the scaling runs
`randomSeed`	one value per repetition, which with the fixed time step reproduces a Mall A run exactly but not a corridor or Daily Urban Life run (Appendix A.1)
`accurateTime/ runDurationSimMinutes`	long enough for late rules to run (230 for the shopping centers, 270 for Daily Urban Life, 200 for the evacuation and 15 for the corridors)
`autoSelectFromMap`, `serviceStationTag`, `serviceStationCapacity`	capacity tests in Mall A: auto-selection off, the tag set to the station type under test (desks, counters or tables), 1–3 places
`infrastructurePlanning/ runDurationSimMinutes`	160 or 200, to cover the whole mall schedule
`dailyUrbanLife/scheduleFile`	generated `DailyUrbanLifeSchedule_x<N>.xml` files for scaling runs
`infrastructurePlanning/ scheduleFile`	the lunch-rush schedule (Section 5.4.2), and a generated `MallA_x50` schedule for the runs with copied people (`Agent_k` uses the rules of `Agent_(k mod 10)`)
clock start, validation corridors	00:00, because their schedules start at midnight
command line	`-UseFixedTimeStep -FPS=30` (one simulated second per frame) in every unrendered run, `-FPS=120` (0.25 seconds) for the step check, and `-NoBlend` for half of the blend runs (Table B.1)

Table 9: Settings changed for the experiments.

4.4 Summary

The first approach showed that purpose cannot come from categories of places and a single flock. This led to rules that name the person and a start time, to stations with capacities, queues and service times, and to path-guided movement that the boids only bend. The current simulator keeps the schedules, actions, stations, environment, clock and scenario managers apart, so that a schedule only states who does what and from when, while deferral, reservation, queueing, service and release decide when each activity starts.

Chapter 5

Evaluation Based on Use Cases

5.1 Introduction

We apply the framework of Chapter 4 to five scenarios and report what the recorded runs measured: a daily urban routine, two shopping centers, an emergency evacuation and a set of directed-flow corridors. All scenarios use the same implementation of a person, schedule parser, movement layer and station model, and they differ only in the schedule file, the map and a small manager that prepares the map. The scenarios show what the rule language can express with little new code, as only Mall B needed a new action, `Interact`, with its button and gate. We also use them to make preliminary measurements of the waiting at shared resources, of the drift between authored and realized times and of how many people one desktop computer can animate. No test compares the simulated people with real pedestrians, but the runs did expose a number of edge cases, which we report where they affect the numbers. Section 5.2 describes the experimental setup, and Sections 5.3 to 5.6 evaluate the daily urban life, shopping center, evacuation and directed-flow use cases. Section 5.7 then measures performance and scalability, Section 5.8 discusses the threats to validity and Section 5.9 concludes the chapter.

5.2 Experimental Setup

Time modes. Every behavioral result in this chapter uses Accurate mode, in which walking and the clock advance together. Visual-mode timings, in which a 100 m walk counts as about 50 simulated minutes, are not valid as timings, and they appear only in the rendered scaling runs of Section 5.7, where only the cost of each frame matters.

5.2.1 Machine, modes and settings

All runs used one desktop computer with an AMD Ryzen 7 7800X3D processor (eight cores), an AMD Radeon RX 7900 XTX graphics card (24 GB) and 32 GB of DDR5-6000 memory. The game ran from the Development editor binaries of **UE** 5.4 (`UnrealEditor.exe -game`) rather than a packaged Shipping build, and the navigation mesh was generated when each level loaded. Behavioral runs used `-nullrhi`, which skips rendering, and only the scaling runs (Section 5.7) were rendered, in a 1920×1080 window. Vertical synchronization was off in the rendered runs, so the monitor did not cap the frame rate (Table B.1). The people are MetaHuman characters with skeletal animation and hair strands. As in the development configuration, `enableInfrastructureAnalysis` was on in every run, so idle people stopped instead of flocking, queued people waited without a time-out, and a failed path left the person stopped with their reservation (Table 9). The boids therefore act in only part of these runs, because idle flocking was off and the blend steers scheduled walks but not walks to a reserved station, queued people or the evacuation, where people follow their paths.

Every behavioral run used Accurate mode (Section 4.3.7), in which walking and the clock advance by the same simulated time in each frame. The unrendered runs also used the engine's fixed time step (`-UseFixedTimeStep -FPS=30`), so every frame was exactly one simulated second, regardless of how fast the machine produced frames. A shopping-center run of about three simulated hours therefore took about half a minute, and a Daily Urban Life run about three minutes. To check how much the results depend on that step, we repeated the evacuation and a subset of the shopping-center and Daily Urban Life runs with 0.25 simulated seconds per frame (`-FPS=120`, Section 5.8.2). The scaling runs measure how many people can be watched interactively, so they were rendered in real time in Visual mode, the mode for viewing. People walk at 200 cm/s by default and at 170 to 230 cm/s in Daily Urban Life. We did not calibrate these speeds, which are faster than the 1.34 m/s of a free pedestrian (Weidmann, 1993), so in Accurate mode a 100 m walk lasts about 50 simulated seconds, roughly a third less than a real pedestrian would need. Three time-outs do not go through this conversion and still count real seconds (Section 4.3.7): the GoTo time-out ends a walk after 90 simulated minutes, the queue time-out after 25, and an exit serves one person every 45 simulated seconds. The queue time-out is unused here, because queued people waited without a time-out, and we note every result that the other two constants affect. An earlier round of runs in Visual mode was superseded by this one for every behavioral result.

The runs used a test branch that adds to the main branch the recorder and the stress test of Appendix B, together with four adjustments for steps of one simulated second, in which a person moves about 2 m. The path follower accepts a waypoint that the person has walked past within two frames of

travel, and the final walk onto an evacuation slot or into the safe zone is shortened so that the person stops inside the slot window. Needs also advance in simulated minutes, and the movement component substeps every 0.05 simulated seconds so that its ledge checks follow ramps. The same branch also lets the capacity setting match station identifiers, so that it reaches the food counters and tables, keeps the clock running when a run meets its end condition before the export timer fires, so that the mall export is still written, and starts the evacuation summary before people react. The results therefore reflect this branch rather than the main branch alone. The public repository (Armada, 2026) holds the framework on its main branch and, beside it, this test branch with its batch scripts and run configurations, while the recorded run files are not public.

Runs start at 07:30 so that the navigation mesh is ready before the first path request, except the evacuation, which starts at the default of 08:00, five minutes before its only rule. The shopping-center runs were capped at 11:20, and most of them ended at about 10:30, when every rule had been consumed and no person was busy, while Daily Urban Life ran until 12:00 (seeds 1 to 5). The evacuation ran three times per population (labeled r1 to r3), and the scaling tests ran once or twice per population. The police-and-robbers and leadership layers were off in every test, because both defer schedule rules. A seed initializes the engine's global random stream, but with a variable frame time the number of draws depends on the real frame timing, so a seed did not make a run repeatable, and Appendix A.1 checks whether it does with the fixed time step. We treat seeds as labels for independent runs in every comparison. With the fixed step, few random inputs differ between seed labels, and in Mall A the main one we found is the offset of up to 80 cm in each direction at which people are placed at their entrances. Runs therefore differ little, and small differences can reach low p-values.

5.2.2 What was recorded

A run recorder, enabled with `-RunRecord=<label>`, writes four files: `rules.csv` holds every consumed rule with its person, authored time, eligibility, dispatch minute and outcome, `stations.csv` every station event of Fig. 17 and `trips.csv` every scheduled walk. `timeline.csv` holds the frame, game-thread and **GPU** times and the state of every person once per second, which is per simulated second in the fixed-step runs and per real second in the rendered runs. The evacuation manager writes its own summary, and a stress-test subsystem (`-StressAgents=N`) clones people and measures frames. The analysis scripts listed in Appendix B.2 produce every number in this chapter and in Appendix A. Comparisons use a two-sided Mann-Whitney U test without correction for the number of comparisons, and the 95% intervals are t-intervals of the mean. A *station visit* runs from a request for a station, admitted or

queued, to its end, and the *wait per visit* is the total queueing time divided by the number of visits, in simulated minutes. Visits that never queued therefore count with zero, and the medians we report are medians across runs of this per-run mean. A walk *arrived* when its GoTo action completed, which happens when the person stops within 180 cm of the target, when movement stops for another reason, or when the GoTo time-out ends a walk within 180 cm, so arrival counts are an upper bound. Values for Mall A that pool runs combine the 45 capacity runs (nine settings, five seeds) with the five runs of Mall A as authored, 50 runs in all. These pooled values use the first five-seed round of the capacity experiment and not its extension to 30 runs per setting (Section 5.4.1). A rule *started late* when it was dispatched after the minute in which it became due, and each late minute is attributed to one *cause* by precedence: queueing, walking to a reserved station, being served (finishing a previous activity), walking to a scheduled target and other. Lateness stops at dispatch, so it leaves out the time that a dispatched station rule then spends in the queue and walking to its station. For station rules, we therefore also split the whole *start delay* of Fig. 12, $s_r - e_r = \ell_r + q_r + w_r + p_r$, where q_r runs from the request to the reservation, w_r from the reservation to the arrival, and p_r is the positioning before service (Section 5.4.3).

5.3 Daily Urban Life

The schedule describes a working morning for 50 people in 304 rules between 08:04 and 10:22. `Agent_0` to `Agent_24` each own a desk: they rest at home, walk to one of seven work areas, work 15 minutes, take a break, work another 15 minutes, and walk home. `Agent_25` to `Agent_49` follow the same pattern 15 minutes later on the same desks (Listing 4.1 in Chapter 4).

The schedule never says how people get anywhere, as `MoveTo` names a tag and the movement layer finds a path, and the station decides whether a `UseStation` request is admitted or queued. The `Break` trigger and a home-going `MoveTo` are exempt from station deferral and release the person's station immediately. The map holds 25 homes, seven work areas (`Work0` to `Work6`, four desks each except `Work6`, which has one), beds, plaza actors and sidewalks that every path prefers (Fig. 18), and people walk at 170 to 230 cm/s. Home and plaza targets receive a small per-person offset, but work areas do not, so all people bound for a work area walk to the same point and count as arrived within 180 cm of it.

People leave home in a staggered stream from 08:22, and with walking at its real pace, 97% of the 304 authored rules were dispatched and 3% were dropped as more than 45 minutes overdue. None were still pending at the end, and 3 to 11% of the dispatched rules started late (Table 10). A dispatched station

rule can still queue or be given up at a break, and 410 of the 515 station rules of the five runs started their service (Table A.3).



Figure 18: Daily Urban Life map during the morning in Visual mode, where each figure is a person following their own schedule and the validation panel is on the left.

Outcome	Percent of the authored rules, walks or visits
Authored rules dispatched	97
Authored rules dropped as too late	3
Authored rules never reached	0
Walks that reached their target	94
Station visits whose reserved place was left before arrival	20

Table 10: Daily Urban Life outcomes with 50 people as a share of the authored rules, of the walks or of the station visits (mean of five runs).

The schedule leaves enough time for walking, as `Agent_0` has 18 simulated minutes between leaving for `Work0` and sitting at their desk, and the walk is about 150 m long. Two things delay rules: competition at shared places and long walks to crowded target points. Two shifts share 25 desks, and of the 481 desk requests in the five runs, 260 queued and 300 ended in a completed work session, and these counts overlap because a request that queued can still end in a completed session. The wait per station visit averaged 1.7 simulated minutes, and queueing was 34% of the start delay of station rules (Fig. 23). A desk queue can outlast the time until the person's break, and since the break releases the place as designed, 7 to 32 reserved stations per run were left before the person reached them. The scenario therefore measures

competition for work sessions, which the time scale of Visual mode had hidden, because there a walk took longer than the schedule allowed for it.

Single-point targets still take time (Fig. 19), as 75% of the walks to `Work3` arrived, against 90 to 100% at the other work areas. The median walk to a work area took 1.6 simulated minutes, close to the 1.3 minutes that 150 m at 2 m/s needs, but the walks to these shared single points have a long tail. Of the 235 walks that arrived, 43 took more than 5 minutes while the person milled about a crowded point, and 7 of them, like all 15 walks that failed, lasted until the GoTo time-out. A person who keeps being pushed around near a crowded point without coming to rest, or who rests just outside the arrival radius, ends their walk only at that time-out. The walk then counts as arrived if the person is within 180 cm and as failed otherwise, and the time-out lasts 180 real seconds, which Accurate mode does not convert and which the clock counts as 90 simulated minutes. These long walks raise the mean walk to a work area to 7.5 simulated minutes, and these walks account for 95% of the late minutes. Without the 7 walks that ended on the time-out, the mean is 5.0 minutes, so the time-out adds about a third to the mean, while crowding at the single points causes the rest of the tail. A few people never left home, because no sidewalk path started where they stood, and overall, 94% of walks counted as arrived, an upper bound by the definition of Section 5.2.2.

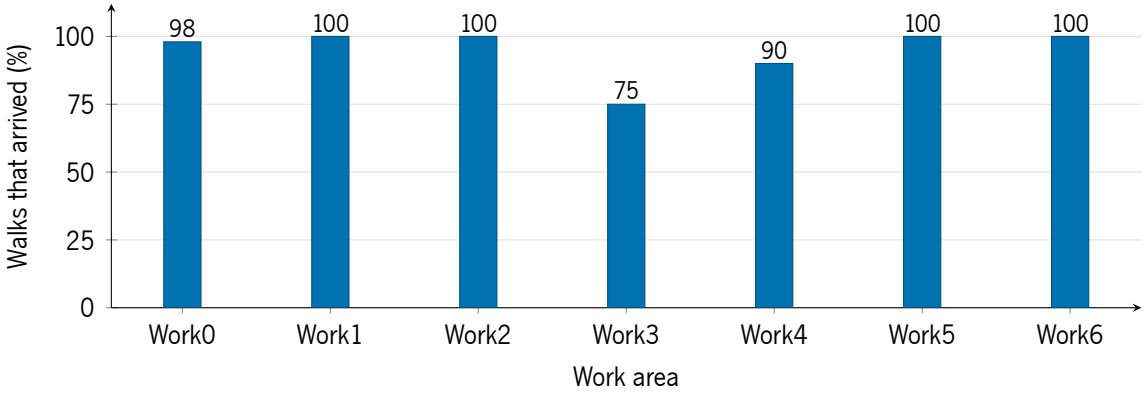
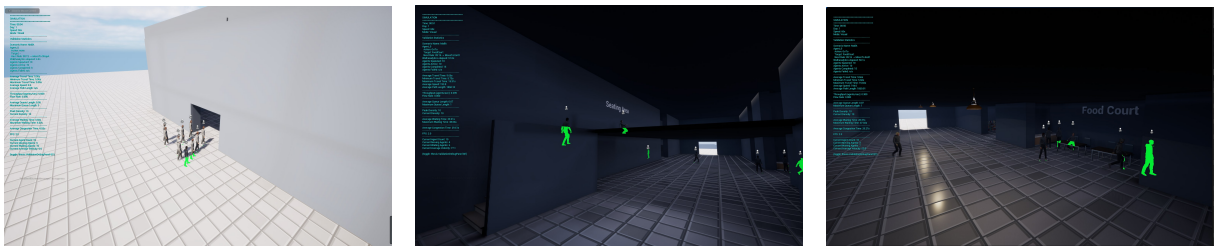


Figure 19: Share of walks that arrived at each work area (five runs).

5.4 Shopping Center: Mall A and Mall B

Each shopping center has its own schedule for ten people: 53 rules in Mall A, of which 28 are `MoveTo`, 15 `UseStation` and 10 `Idle`, and 59 in Mall B, which adds four walks, one station use and the single `Interact`. Every person waits at an entrance, browses one or two shops, uses a shop desk or a seat, eats, and leaves. In Mall A, nine people eat, and the meal is a two-stage pipeline: a food counter for 10 minutes, then a food table for 14 (Listing 4.2 in Chapter 4). The table is chosen only when the counter

stage ends, so a slow counter delays the table request. Figure 20 shows the entrance of Mall A and its levels, as the map has more than one floor.



(a) The entrance (b) Its levels (c) The food court

Figure 20: Mall A, with (a) its entrance, (b) a view of its levels and (c) its food court, where a meal uses a counter and then a table.

Mall B keeps the same ten people and their schedules with three changes: four people eat at a second food court, service points have two places instead of one, and `Agent_0` first walks to a button (`MoveTo` at 08:04) and presses it (`Interact` at 08:08). The button opens a second entrance, where the odd-numbered people start (Fig. 21).



(a) The two entrances (b) Walking to the button (c) The interior

Figure 21: Mall B, with (a) its two entrances, (b) a person walking to the button that opens the closed entrance with the `Interact` action, and (c) its interior.

Both maps have two shops with desks, a seating area, and a food court built as a super station with four counters and four tables (Fig. 20c). Mall B adds `FoodCourt2`, the button, and a gate that removes its area from the navigation mesh while closed. The infrastructure manager places people at the entrances and sets the capacity of every station whose actor tag or identifier matches a configured name, so that the capacity tests can change one kind of station at a time. A place is reserved when the rule fires, and a person who finds every station of the requested type full joins the queue of the first one and waits where they stand. The walk to the food court took 0.4 simulated minutes on average in Mall A, well within the 10 minutes that `Agent_0`'s schedule allows.

5.4.1 Capacity experiments

We kept the Mall A map and schedule fixed and changed one kind of station at a time (shop desks, counters or tables) to one, two or three places, with five runs per setting, later extended to 30 runs per setting (Fig. 22). In the counter and table tests, the shop desks kept their editor capacity, which allowed two people at one desk, so the one-place bars of the three groups are not the same configuration, although their medians coincide.

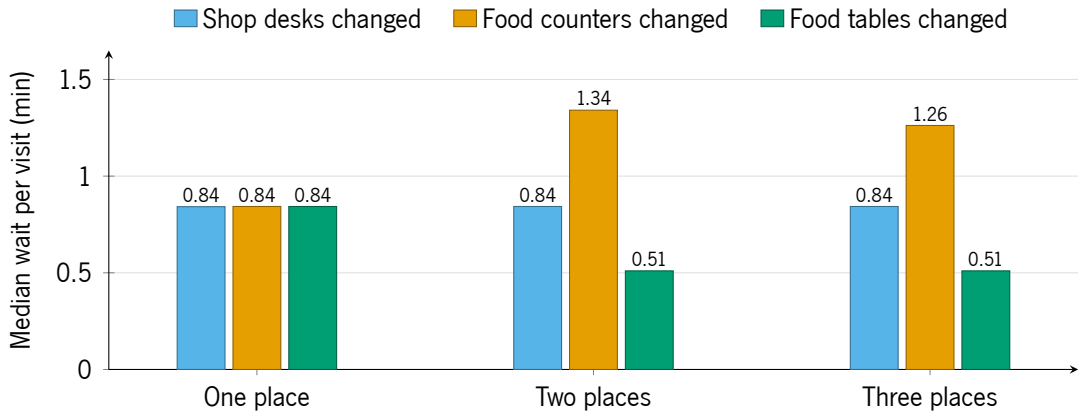


Figure 22: Median wait per station visit in Mall A by number of places (simulated minutes, 30 runs per setting, lower is better), with the full statistics in Table A.2.

The one-place baseline differs between the two rounds, at 0.81 simulated minutes over the five seeds and 0.84 over the 30 runs, so each value below is quoted with the round it comes from. In the first five runs per setting, two or three places at the tables lowered the wait per visit from 0.81 to 0.51 simulated minutes in all five runs ($p = 0.008$). More places at the counters raised it to 1.21 with two places ($p = 0.008$) and 1.35 with three ($p = 0.10$), and the queue joins explain why. With one place per counter, 10 counter requests and 5 table requests queued over the five runs. With two or three counter places, the counter queues disappeared, but more diners reached the tables at the same time, and as the tables kept one place each, 10 and 11 table requests queued, so the bottleneck moved down the pipeline and the wait grew. With one table place, only 5 of 45 table requests queued in five runs, about one queued visit per run, so the lower wait with more table places is the removal of that visit. No person ever joined a desk queue, and the desk settings made no difference ($p = 1.0$). Over 30 runs per setting (Fig. 22 and Table A.2), the medians show the same pattern: 0.84 minutes with one place, 1.34 and 1.26 with two and three counter places ($p < 0.001$), 0.51 with two or three table places ($p < 0.001$) and 0.84 for every desk setting ($p = 0.89$). The table result is a difference of about 20 seconds per visit, small against a 24-minute meal, so what matters is its direction, which behaves as a tandem queue should and which the lunch rush below repeats with waits of several minutes. The mean with three counter places (2.43 minutes) is not a valid

summary, because in 3 of its 30 runs a diner was admitted to a table they never reached and held it until the run ended, with one or two people queued behind them. These runs averaged 7.5 to 16.0 minutes of wait per visit, while the other 27 runs averaged 1.25 minutes (Section 5.8.1).

The logs also explain why queues persist while equivalent stations are free. All queue joins of a given kind went to a single station, because a queued person stays with the first matching station even when an equivalent one frees up. The capacity comparisons therefore hold under this selection policy, and one shared queue per area could change their sizes. A place is also held while its owner walks to it, though briefly at the real walking pace: in Mall A, a person reserved a counter on average 0.3 minutes and a seat 0.6 minutes before reaching it.

5.4.2 A lunch rush

Ten people seldom compete, so we also ran a lunch-rush variant of the Mall A schedule, with ten runs per setting (Table 11). In this variant, the schedule sends all nine diners to the food court at 08:40 and has them request a counter at 08:50, they leave no earlier than 09:20, and the other rules are unchanged. We did not add people by copying them, because with 50 copies of the authored schedule the shared single-point targets jammed. In five such runs, only 26 to 77 of 87 to 107 walks arrived, 148 to 200 of the 265 rules were dispatched and 11 to 41 station services were completed.

Stations changed	Places	Median wait	Range over runs	Mean wait (SD)	Queued visits per run	Start delay	Start delay spent queueing (%)	p
Food counters	1	4.8	2.9–4.8	4.6 (0.6)	4.8	7.6	92	–
	3	5.9	5.3–31.5	8.6 (8.1)	4.0	0.7	0	<0.001
Food tables	1	4.8	2.9–4.8	4.6 (0.6)	4.8	7.6	92	–
	3	4.6	2.8–4.6	4.4 (0.6)	3.9	7.6	92	0.002

Table 11: Wait per station visit and start delay in Mall A during a lunch rush with ten runs per setting, in simulated minutes, where SD is the standard deviation and p compares three places with one, and where the start delay covers the counter stage only, so a queue that moved to the tables appears in the wait per visit and not in that column.

In every rush run, one of the nine diners could not get within the arrival radius of the crowded food-court point, and their walk timed out before it reached the counter stage, so a run records eight counter requests rather than nine and each setting 80 over its ten runs. With one place per counter, 39 of the 80 counter requests queued, the wait per visit averaged 4.6 simulated minutes (median 4.8), and queueing was 92% of the start delay of station rules. Three places per counter removed the counter queues but moved them to the tables, which kept one place: 40 of 80 table requests queued, later pipeline stages queued for 14.9 minutes on average, and the wait per visit rose to 8.6 minutes (median 5.9, $p < 0.001$). Three places per table lowered the wait only slightly (4.4 minutes, median 4.6, $p = 0.002$), because the

counters, which kept their editor capacity in the table tests, stayed the bottleneck, with 39 of 80 counter requests queued and none at the tables. Under contention, extra places at one stage of the pipeline moved the queue to the next stage that was short of capacity.

5.4.3 Where late minutes come from

Late starts were rare, as between 0% and 4% of the dispatched rules started late in the capacity settings, and the few late minutes in Mall A were mostly spent finishing a previous activity rather than walking. Lateness stops at dispatch, however, so Fig. 23 also splits the whole start delay of station rules. In Mall A, a station rule started its service on average 1.0 simulated minute after it became eligible, with no lateness, 0.7 minutes queueing for a place and 0.3 minutes walking to the reserved station. Queueing was 67% of the start delay over all 50 Mall A runs and 76% in Mall A as authored, and later pipeline stages, such as the table after the counter, queued for 1.2 minutes on average. The start delay covers only the first stage of a pipeline, so with two or three counter places it shows no queueing, while the table stage queued for 3.2 to 3.6 minutes on average. The 105 Daily Urban Life station rules that never started are also left out of its mean, and in Mall B, where no person queued, the start delay was the walk alone. With walking at its real pace, waiting for shared stations is the largest part of the gap between authored and realized times in the shopping centers, while in Daily Urban Life long walks to crowded target points take a larger share (Section 5.3).

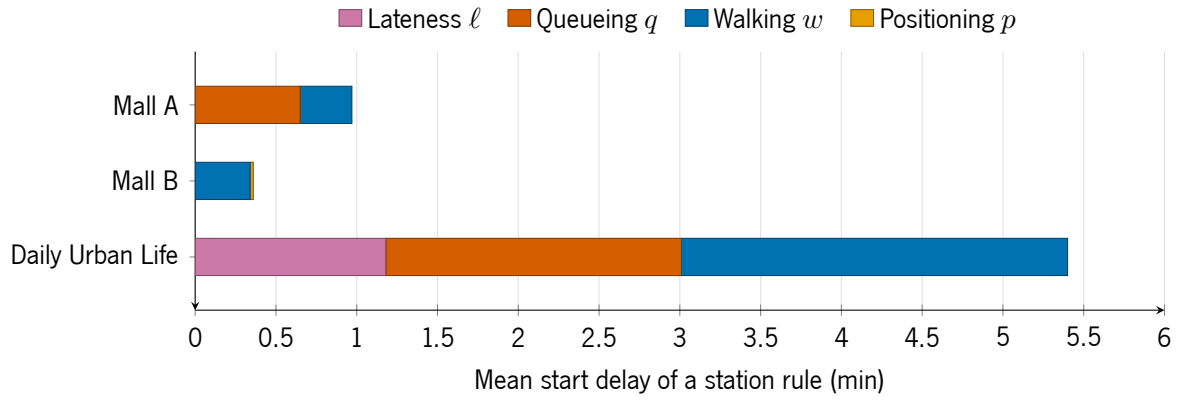


Figure 23: Start delay of station rules split into its parts (simulated minutes), with the values in Table A.3.

5.4.4 Mall A against Mall B

In Mall B, no person ever queued, and its rules started on time about as often as in Mall A, but because schedule, layout and capacities all differ, the comparison does not isolate the effect of layout. In Visual mode, these runs had exposed an edge case of the reservations, as a reservation has no time-out and a

failed path only stops the person, so a person who never reached their reserved station held the place until the run ended. In Accurate mode, no station visit was still open at the end of any layout run, and no walk failed, so this edge case did not affect this comparison. It can still occur, and it dominated three capacity runs with three counter places (Section 5.4.1), in which a person admitted to a table never reached it.

5.5 Emergency Evacuation

The evacuation needs one rule (Listing 5.1), and the rest is scenario code. The emergency manager starts a fire incident covering the map, raises the alarm for every person and starts a countdown authored as 120 seconds, which keeps its Visual meaning of 120 simulated minutes in Accurate mode. Each person first waits for a reaction delay, authored as 0 to 5 seconds, which Accurate mode turns into 0 to 5 simulated minutes, long for a reaction to an alarm but short against the 120-minute countdown. The person then chooses an exit by a score that combines distance and queue length, asks for a queue place and walks to their slot, and may switch exits every 5 seconds, at most three times.

```
<agent id="Agent_0">
  <rule time="08:05"><then action="StartEvacuation"/></rule>
</agent>
```

Listing 5.1: The evacuation schedule. The trigger is global and idempotent, so one coordinator is enough.

The map is an open floor with walls, two exits, and a safe zone (Fig. 24). Each exit lets one person through every 1.5 real seconds, a constant that Accurate mode does not convert, so an exit serves one person every 45 simulated seconds, far fewer than a real door of that width would pass. The results below therefore test the exit and queue mechanism rather than an evacuation time, and they cannot yet say whether an exit would become a choke point, the question that Chapter 1 opens with. Each exit lays out its queue as slots behind it, at most 25 on this map (the recorded peak at both exits in every run with 100 or 200 people). A person who has passed an exit walks to the safe zone and is counted as safe within 200 cm of its center, and when the countdown ends, every other person becomes a casualty.

We ran the evacuation with 50, 100, and 200 people, three times each, made the larger populations by copying the map's people, and repeated every run with a step of 0.25 instead of one simulated second per frame (Fig. 25). At one second per frame, all 50 people passed an exit before the countdown ended, 89 to 100 of 100, and 44 to 75 of 200. At 0.25 seconds per frame, all 50 and all 100 did so in every run, and 57 to 66 of 200.

An exit lets one person through every 45 simulated seconds, so in the 120-minute countdown it could pass 160 people, and two exits 320, but the exits passed at most 51 people each, so their designed

capacity does not explain the losses. The losses came from stalled exits: in one run with 100 people and one with 200 at one-second steps, one exit passed at most two people after its queue stalled, and with 200 people both exits passed fewer than 40 at either step. We did not check the positions of the people, but the pattern suggests an edge case of the exit queues. People who cannot get a queue place keep pressing toward the exits, so the person who becomes first in a queue cannot always reach the head slot. Only 10 to 31 people per run were counted as safe, because the first arrivals stop at the rim of the safe zone and the rest cannot enter. Everyone else counts as a casualty, including people who had left the building, so we report passing an exit rather than casualties, and we treat the stalled exits and the blocked safe zone as edge cases of the scenario (Section 5.8.1, per-run counts in Table A.4).



Figure 24: Evacuation shortly after the alarm, with the fire along one wall and the people heading for the exits.

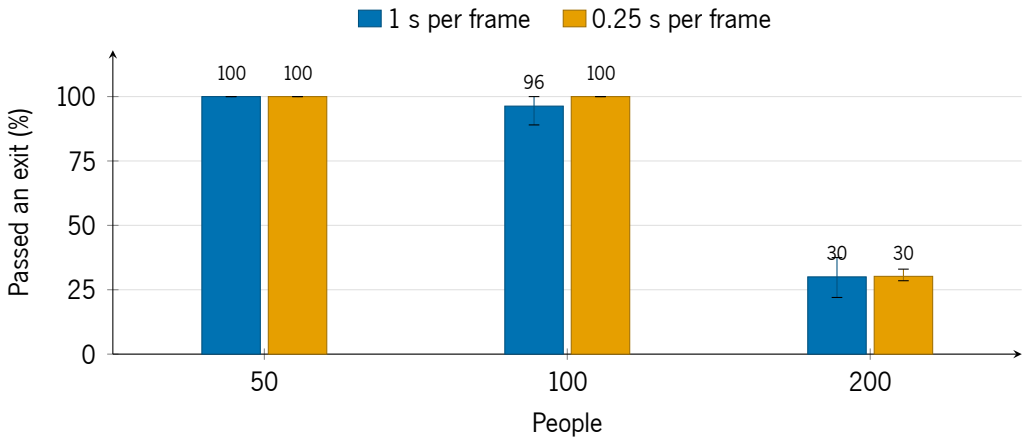


Figure 25: Share of people who passed an exit before the countdown ended (mean of three runs, error bars show the range), with every run in Table A.4.

5.6 Directed-Flow Test Corridors

Seven small maps test movement on their own: a corridor walked in one direction (V01) and in both (V02), a corner (V03), a bottleneck (V04), a T-junction (V05), a functional area whose twenty occupants all leave through one designated exit (V06) and two crossing diagonal flows (V07). Each map holds one or two cohorts written as super groups, whose members each receive a copy of the group's rules, and an on-screen panel reports flow, density, waiting and travel time. In the bottleneck map, for example, ten people idle for two minutes and then walk to `Exit_Right`.

Each cohort walks to a single exit point with a 180 cm arrival radius, so runs finish with the cohort bunched around it, as at the work areas (Fig. 19), and the panel's arrival counts depend on the target rather than on the corridor. Several panel measures are also proxies, as its density, for example, is the number of people on the map. We therefore used three of the seven maps for a single test of the hybrid blend, the one-way corridor, the two-way corridor and the bottleneck, because their straight flows make spread and speed easy to measure. The other four maps remain demonstrations of cohort rules.

5.6.1 The hybrid blend on and off

The blend is active in Accurate mode on scheduled walks, but not on walks to a reserved station, in a station queue or during the evacuation, so it shapes only part of the walking in this chapter, and here we isolate it. We ran the one-way corridor (V01), the two-way corridor (V02), and the bottleneck (V04) three times each with the blend and three times with `-NoBlend`, which leaves plain path following, and recorded every person's position once per simulated second (Table 12). The measures cover people still more than 6 m from their cohort's end point: the share of these position samples whose capsule (radius 34 cm) intersects another person's, the spread of each cohort across its direction of travel (the standard deviation of lateral positions), and its speed along that direction. The last columns give the time from a trip's start to within 4 m of the end point and the number of people who got there, where the end point is the cohort's centroid at its last sample rather than its target. Speed is sampled only more than 6 m from the end point and time covers the whole trip, so the two can disagree, as in the one-way corridor, where the blend lowered the speed but not the time to the end.

With the blend, the cohorts spread wider in the one-way corridor (154 against 144 cm) and in the bottleneck (128 against 103 cm), while in the two-way corridor the spread was about the same (120 against 123 cm), as were the arrivals (55 against 57 of 60). In the bottleneck, the cohorts also reached their end point sooner (37 against 45 s), and more people got there (57 against 53 of 60). Overlaps were

rare in every setting, at most 3% of position samples in any run. Samples are one simulated second apart, so short overlaps between samples are missed. With three runs per setting, and with corridor runs that a seed does not reproduce (Appendix A.1), these are directions, not sizes: the blend spread the cohorts in two of the three maps and helped where people converge, and it does not prevent overlaps.

Map	Blend	Overlapping samples (%)	Spread (cm)	Speed (cm/s)	Time to end (s)	People that got there
Corridor, one way	on	1 (0–2)	154	153	23.0	29/30
	off	2 (2–3)	144	164	23.2	30/30
Corridor, both ways	on	0 (0–0)	120	23	26.0	55/60
	off	1 (0–2)	123	19	28.2	57/60
Bottleneck	on	0 (0–0)	128	29	37.1	57/60
	off	0 (0–0)	103	14	44.7	53/60

Table 12: Hybrid blend on and off in three test maps with three runs each, where overlaps show the mean with the range over runs in parentheses, spread is measured across the direction of travel and speed along it, as the median over runs.

5.7 Performance and System Scalability on One Machine

We raised the population of the evacuation and of Daily Urban Life from 50 to 1600 people. The stress-test subsystem copies the level's people at start, and in Daily Urban Life, each copy received the rules of one of the 50 original people, so homes and desks were shared by more people as the population grew. After 30 seconds of warm-up, it recorded 120 seconds of frames in Visual mode, the mode for watching (Section 5.2.1), with vertical synchronization off, twice per population up to 400 people and once at 800 and 1600 (Fig. 26).

Taking 30 frames per second as a common interactive threshold, the evacuation stayed above it on average with 100 people (32.8), although its 95th-percentile frame times of 51.4 and 47.0 ms correspond to 19 to 21 frames per second. Daily Urban Life reached the threshold with 50 people (37.6) but not with 100 (16.9). Between 50 and 800 people, the frame time grows roughly in proportion to the population in the evacuation and faster than that in Daily Urban Life, with fitted exponents of about 0.9 and 1.2 on logarithmic axes. The limit is the update of the people on the processor, as in every run the game-thread time equaled the frame time to within rounding. Between 50 and 800 people, the render thread needed 3.9 to 43.6 ms on average, at most a quarter of the frame time (in Daily Urban Life with 50 people), and the **GPU** needed at most 40.6 ms per frame on average, in Daily Urban Life with 800 people (Table A.5). Neighbor queries, boids forces, sonar and wall probes and synchronous path queries all run on the game thread. The faster growth in Daily Urban Life is consistent with its shared destinations putting

more neighbors around each moving person as the population grows, although we did not confirm the cause with a profiler. Each character also has a fixed cost, as after the evacuation countdown, when no person moves, a frame still took 0.17 to 0.22 ms per person. Below 30 frames per second, a rendered run in Accurate mode covers more than one simulated second per frame, which can change its results (Section 5.8.2), so the batch runs used the fixed step.

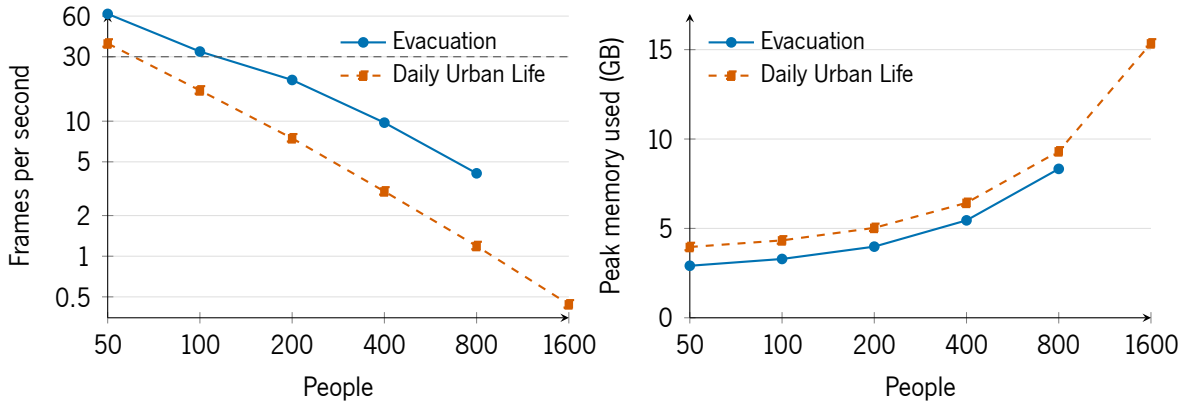


Figure 26: Frame rate (left) and peak memory (right) by number of people in Visual mode, where the dashed line marks 30 frames per second and the Daily Urban Life point at 1600 people is not comparable with the others, while the evacuation has no point there because it crashed in its first frame (every run in Table A.5).

We repeated each population once with `-StressHideMeshes`, which hides the MetaHuman meshes and stops their animation and hair, to separate the framework from the characters (Table 13). In the evacuation, the characters took 49 to 67% of the frame time, and without them, a frame took 12.7 ms with 100 people and 33.8 ms with 400. In Daily Urban Life, they took only 16 to 34%, and the framework alone needed 19.0 ms for 50 people and 38.8 ms for 100. The people's own update, with neighbor queries and synchronous path queries at shared destinations, is the limit.

Scenario	People	Characters shown (ms)	Characters hidden (ms)	Frame time spent on characters (%)
Evacuation	50	16.1	8.2	49
	100	30.6	12.7	58
	200	50.3	16.8	67
	400	102.8	33.8	67
	800	243.4	94.8	61
Daily Urban Life	50	26.6	19.0	28
	100	59.2	38.8	34
	200	133.7	100.7	25
	400	331.2	276.3	17
	800	842.5	711.2	16

Table 13: Mean frame time with the MetaHuman characters shown and hidden, and the share of the frame time that the characters took.

The cost also changes during a run (Fig. 27), as with 400 people, the evacuation reached its lowest frame rate about 15 seconds in, when all people chose exits and planned paths at once, and recovered when the countdown ended. Daily Urban Life, in contrast, slowed steadily as more people set off.

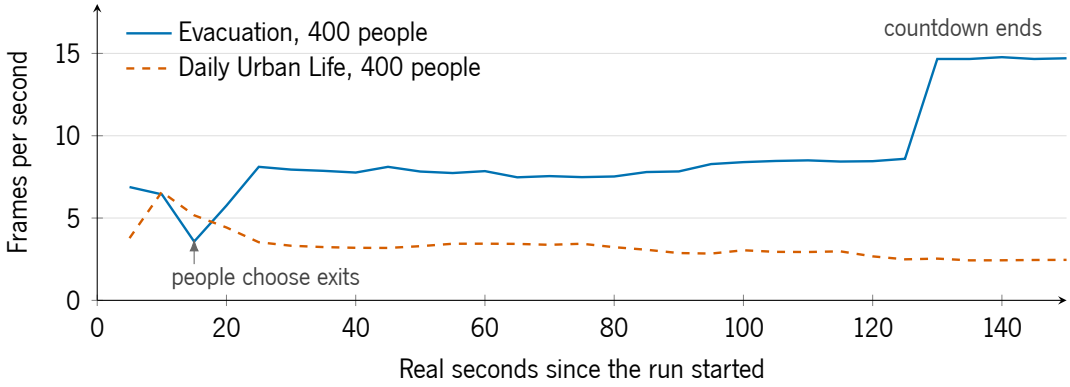


Figure 27: Frame rate during one run of each scenario with 400 people (5-second bins).

Between 50 and 800 people, memory grew by about 7.4 MB per person in the evacuation and 7.3 MB in Daily Urban Life, from about 2.6 GB and 3.6 GB, so the 32 GB of the machine was not the limit. The evacuation with 1600 people crashed in its first frame with an assertion in the engine's render graph and the hair-strands module in the call stack. Daily Urban Life ran with 1600 people at 0.44 frames per second, and at such rates the clock falls behind real time and movement takes large steps, so behavior at the largest populations is not comparable with the smaller ones.

5.8 Threats to Validity

With the fixed time step, a seed reproduced a run exactly in the Mall A pairs we compared, but not in the corridors or in Daily Urban Life (Appendix A.1). Mall A runs with the same seed label under different settings therefore share their random draws until the settings make them diverge, while the tests treat runs as independent samples, which is conservative, since a paired test over seed labels would have more power. Because these runs differ so little, a low p-value marks a consistent direction rather than a large effect, as the tables show with about 20 seconds per visit. The measures are proxies defined by our implementation: the wait per visit ignores the time a place is held while its owner walks, late minutes are attributed with a fixed precedence and stop at dispatch, and arrival counts are an upper bound, because a walk counts as arrived whenever the walk action completes. The results depend on the frame step (Section 5.8.2), as movement within a frame of one simulated second is coarse, while with 0.25 seconds, more walks stalled and an unreleased reservation dominated one counter run. The edge cases of Section 5.8.1 dominate some runs, and the GoTo time-out and the exit service, which count real

seconds, lengthen some Daily Urban Life walks and slow every exit of the evacuation. People walk faster than free pedestrians, the scaling runs were measured in Visual mode rather than in rendered Accurate mode, only three of the seven corridor maps were measured, and larger populations were obtained by copying people. All performance results come from one machine, a Development editor build and one set of MetaHuman characters, and the runs without characters (Table 13) are single runs, so the character shares are estimates. No run was compared with observed pedestrians, so a reduction in simulated waiting describes the model, not a real building.

5.8.1 Edge cases and their effect on each result

The runs exposed five edge cases of the implementation, and Table 14 states for each one which runs it affects and whether it changes the direction of a result or only its size.

The direction of every capacity result, and the medians that report it, survive these cases, but the mean with three counter places does not, so its median of 1.26 minutes is the value to use. The same holds in the lunch rush, where one undiagnosed run with three counter places averaged 31.5 minutes.

Edge case	Runs it affects	What it changes
A reservation has no time-out, so an admitted person who never reaches the place holds it	3 of the 30 Mall A runs with three counter places, and one counter run at 0.25 s per frame	The mean wait with three counter places (2.43 minutes, against 1.25 in the other 27 runs), but not its median (1.26) or the direction of any capacity result
A queue stays bound to the first matching station	Every Mall A run with a queue, including the lunch rush	Which station queues and the size of the waits, while the direction held in every comparison
Walks end at single target points, and a person who cannot settle there ends their walk only at the GoTo time-out	Daily Urban Life, one diner in every lunch-rush run, and the Mall A runs with copied people	The mean walk to a work area (7.5 minutes, 5.0 without the 7 time-out walks) and the size of the mall populations, which jammed when copied
Exit queues can stall	Evacuation runs with 100 and 200 people	How many people passed an exit (89 to 100 of 100 and 44 to 75 of 200 at one-second steps)
The first arrivals block the rim of the safe zone	Every evacuation run	The number counted as safe (10 to 31 per run), so we report passing an exit instead of casualties

Table 14: Edge cases of the implementation and the results they change.

5.8.2 Frame step

Every behavioral result above uses one simulated second per frame, so we repeated the counter tests with one and three places (five seeds each), two Daily Urban Life runs and the evacuation with 0.25 simulated seconds per frame (`analysis/step_check.py`). The evacuation improved with the finer step (Section 5.5), while in Daily Urban Life fewer rules were carried out (564 against 591 of 608 in the two

runs, with 44 against 17 dropped as overdue). Fewer walks also arrived (171 against 187 of 200), fewer station visits ended in a completed service (92 against 127), and 29 walks failed, against 13. In Mall A, every rule was dispatched at both steps in all but one run, but one or two exit walks per run had not arrived when the run ended, 21 in the ten runs against 1 at one-second steps. With one counter place, the mean wait per visit rose from 0.81 to 1.16 simulated minutes (median 0.84 at both steps). With three places, it rose from 1.35 to 2.90, mostly because of one seed with the reservation edge case of Section 5.8.1. The direction of the counter result held at both steps, but its size did not, and with only two step sizes we cannot say whether the results converge at finer steps.

5.9 Summary

One implementation of a person ran every scenario of this chapter with only one new action, `Interact`, and queueing was 76% of the start delay of station rules in Mall A as authored and 92% in a lunch rush with one counter place, while long walks to crowded target points took most of the delay in Daily Urban Life. Extra places helped only at the stage of a pipeline that was short of capacity, a direction that held at both frame steps and survives the edge cases of Section 5.8.1.

Chapter 6

Conclusions and Future Work

In Section 6.1, this chapter answers the research question of Section 1.2.1 and assesses objectives O1 to O5 and the functional requirements R1 to R7, before Section 6.2 sets out the future work.

6.1 Conclusions

The main result of this work is what the recorded runs show about a place. Once walking took its real time, waiting for shared stations became the largest contributor to the gap between authored and realized times in the shopping centers, with queueing at 76% of the start delay of station rules in Mall A as authored and 92% in a lunch rush with one counter place, while in Daily Urban Life long walks to crowded single target points accounted for 44%. Relief at one stage of a pipeline also moved the queue to the next one, as over 30 runs per setting extra places at the food tables lowered the median wait per station visit from 0.84 to 0.51 simulated minutes, extra counter places only moved the queue to the tables, and the lunch rush showed the same shift with waits of several minutes. These directions held at both frame steps, and every result can be traced back to the recorded rules, station events and walks, which is also how the few runs that differ from the rest were explained (Section 5.8.1).

Our answer to the research question is therefore a framework in which schedules issue timed requests, a movement layer combines navigation paths with bounded boids steering, capacity-limited stations decide when an activity can take place and the environment constrains both, while a recorder explains every delay. Timing required explicit semantics rather than more rule types, as an authored time is a request that is deferred while the person is busy, consumed on dispatch and dropped when far too late. Reuse is what makes these analyses cheap, as one implementation of a person ran a 50-person daily routine with 304 rules, two shopping center layouts with 53 and 59 rules, an evacuation started by one rule and seven directed-flow corridor maps written as cohort rules. Six actions were enough for all of them, the only new action that a scenario needed was `Interact`, added for the button of Mall B, and the implementation

animates 50 people in the daily routine and 100 in the evacuation at a mean above 30 frames per second on one desktop computer, with slower frames in the tail of the evacuation runs.

6.1.1 Objectives and requirements

01 (authoring language) was achieved: **XML** schedules express per-person and cohort rules and multi-stage station use, with defined deferral, consumption and a 45-minute retry window, and conditions and probabilities are available for schedules that need them.

02 (movement layer) was achieved: paths prefer sidewalks, boids steering turns the person at most 18° from their path, and wall probes remove pushes into walls. In the test maps, the blend spread the cohorts in the one-way corridor and in the bottleneck, where more people reached their end point and did so sooner, while overlaps stayed rare.

03 (shared resources) was achieved, and its result held in direction at both frame steps: extra places help at the stage of a pipeline that is short of capacity, as the food tables ($p < 0.001$) and the lunch rush above show, while desk places made no difference because no desk formed a queue.

04 (reuse) was achieved for every scenario family: all of them run on one implementation of a person, Mall B added a second food court and a button-operated entrance with only a new `Interact` action, a button and a gate, and the remaining scenario-specific code is minimal and easy to identify, in the scenario managers, a few branches of the person and a few map-name settings.

05 (measurement) was achieved: the run recorder and the stress test, on a test branch of the public repository, and the analysis scripts of Appendix B produce every result in Chapter 5 from recorded files, and with a fixed time step a seed reproduced the Mall A runs exactly.

Table 15 assesses the functional requirements of Table 2, which the objectives above serve as mapped in Table 3. A requirement counts as met when the evidence shows what its text asks for, and where a status names a scope, such as the mechanism, the tested scenarios or the authored populations, that scope is the one that the requirement itself sets, so the requirement is met in full within it.

6.1.2 Limitations

This work makes no attempt at calibration, so its numbers describe the model rather than real buildings. The results come from Accurate mode at one simulated second per frame, and their sizes, though not their directions, depend on that step. Three time-outs still count real seconds, which lengthens some Daily Urban Life walks and slows the evacuation exits to one person per 45 simulated seconds, and walking speeds and service durations are authored rather than calibrated. A seed reproduces the Mall A runs

exactly but not the corridor or Daily Urban Life runs, the blend was measured in three of the seven corridor maps, and the malls hold ten people because copied people jam at single-point targets. A few edge cases shape some runs: reservations without a time-out and queues bound to the first station in the station model, and stalled exit queues and a blocked safe zone in the evacuation (Section 5.8.1). Performance was measured on one machine with MetaHuman characters, which take a large share of the frame time. Conditions and probabilities are implemented but unused by the shipped schedules, and no user has yet written a schedule in a study.

Requirement	Status	Evidence
R1 Purpose as authored data	Met for the mechanism	Six actions in **XML** schedules describe every scenario without recompiling, and a study with non-programmers is future work.
R2 Timing under contention	Met	Deferral, consumption and a retry window, no authored minute skipped, and in Accurate mode walking and the clock advance together, while three time-outs still count real seconds (Section 6.1.2).
R3 Shared capacity-limited resources	Met	Reservation, queue, timed service, release and multi-stage activities work, and extra places shorten waits at the bottleneck stage of a pipeline.
R4 Reuse across scenarios	Met for the tested scenarios	Every scenario family and seven corridor maps run on one implementation of a person, with scenario-specific code kept to the scenario managers and a few map-name settings.
R5 Environmental constraints	Met	Sidewalks, walls, gates, exits and safe zones shape outcomes without appearing in rules.
R6 Observability and measurement	Met	Recorder files and the analysis scripts reproduce every result, the recorder and the batch scripts are public, and with a fixed time step a seed reproduced the Mall A runs exactly.
R7 Interactive performance	Met for the authored populations	A mean above 30 frames per second with the 50 people of Daily Urban Life and up to 100 in the evacuation, the fixed-step runs are much faster than real time, and the stress test measures the cost as the population grows.

Table 15: Assessment of the requirements of Chapter 3.

6.2 Prospect for Future Work

6.2.1 Other populations and places

The separation of schedules, stations and movement is not tied to pedestrians in buildings, so the same analysis of waiting and capacity could be applied to other populations with little new code. Boids began as a model of animals (Reynolds, 1987), and a herd or a flock with a daily cycle maps onto the framework, with feeding grounds, waterholes and resting sites as capacity-limited stations. The existing needs component, which already models hunger and energy, would decide when the animals go to them. Animals react to

predators and to each other more than to a timetable, so they would need perception-driven layers in the core code, but the schedule, station and recording layers would carry over. With little or no new code, the framework could also describe hospitals with consultation rooms and waiting areas, airports with check-in desks and boarding gates, theme parks with ride queues, and the daily routines of non-player characters in games, where authored schedules are already common (Bethesda Game Studios, 2011; ConcernedApe, 2016).

6.2.2 Performance

Updating the people on the game thread limits interactive use, and a uniform grid for neighbor search (Reynolds, 2006; Ericson, 2004), asynchronous path queries and level of detail for distant people would reduce its cost. A larger step would move people to **UE**'s data-oriented Mass framework (Epic Games, 2026b,c) or steering to the **GPU** (Owens et al., 2007; Joselli et al., 2009), while simpler characters would lower the MetaHumans' share and avoid the hair-strand crash.

6.2.3 Shared resources and timing

The station model would benefit from reservations that expire when their owner does not arrive and from one queue per area and station type instead of a queue bound to the first station. Arrival areas instead of single target points, exit queues that keep flowing, a safe zone that counts every person who enters it and time-outs expressed in simulated time would also remove the edge cases of Section 6.1.2.

6.2.4 Calibration and validation

Calibration should start with walking speeds (Weidmann, 1993), the directed-flow corridors against recorded fundamental diagrams (Seyfried et al., 2005; Zhang et al., 2011) and the evacuation against the RiMEA test cases (RiMEA e.V., 2025) and ISO 20414 (International Organization for Standardization, 2020). Controlled studies could then vary one factor at a time, such as the number of tables, with larger populations and demand generated from distributions (Ricks et al., 2020).

6.2.5 Authoring tools

An editor tool that checks tags, station types and times against the open map and shows realized times on each person's timeline would show users the gap while they write, and a study with non-programmers would test the authoring claim.

Bibliography

Jan M. Allbeck. CAROSA: A tool for authoring NPCs. In *Motion in Games (MIG 2010)*, volume 6459 of *Lecture Notes in Computer Science*, pages 182–193. Springer, Berlin, Heidelberg, 2010. doi: 10.1007/978-3-642-16958-8_18.

Rui Armada. Masters-Thesis: Boid Animation Rule System. GitHub repository, 2026. URL https://github.com/RuiArmada/Masters-Thesis. Accessed 2026-10-24.

Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975. doi: 10.1145/361002.361007.

Bethesda Game Studios. The Elder Scrolls IV: Oblivion. Video game, Bethesda Softworks, 2006.

Bethesda Game Studios. The Elder Scrolls V: Skyrim. Video game, Bethesda Softworks, 2011.

Eric Bonabeau. Agent-based modeling: Methods and techniques for simulating human systems. *Proceedings of the National Academy of Sciences*, 99(suppl. 3):7280–7287, 2002. doi: 10.1073/pnas.082080899.

C. Burstedde, K. Klauck, A. Schadschneider, and J. Zittartz. Simulation of pedestrian dynamics using a two-dimensional cellular automaton. *Physica A: Statistical Mechanics and its Applications*, 295(3–4): 507–525, 2001. doi: 10.1016/S0378-4371(01)00141-8.

Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, Boca Raton, FL, 2018. doi: 10.1201/9780429489105.

ConcernedApe. Stardew Valley. Video game, 2016.

Sean Curtis, Andrew Best, and Dinesh Manocha. Menge: A modular framework for simulating crowd movement. *Collective Dynamics*, 1:A1, 2016. doi: 10.17815/CD.2016.1.

Alessandro Ribeiro da Silva, Wallace Santos Lages, and Luiz Chaimowicz. Boids that see: Using self-occlusion for simulating large groups on GPUs. *Computers in Entertainment*, 7(4):1–20, 2009. doi: 10.1145/1658866.1658870.

Carlos Delgado-Mata, Jesus Ibanez Martinez, Simon Bee, Rocio Ruiz-Rodarte, and Ruth Aylett. On the

use of virtual animals with artificial fear in virtual environments. *New Generation Computing*, 25(2): 145–169, 2007. doi: 10.1007/s00354-007-0009-5.

Epic Games. Using avoidance with the Navigation System in Unreal Engine. Unreal Engine Documentation, 2026a. URL https://dev.epicgames.com/documentation/en-us/unreal-engine/using-avoidance-with-the-navigation-system-in-unreal-engine. Accessed 2026-10-09.

Epic Games. Mass Entity in Unreal Engine. Unreal Engine Documentation, 2026b. URL https://dev.epicgames.com/documentation/en-us/unreal-engine/mass-entity-in-unreal-engine. Accessed 2026-10-08.

Epic Games. Overview of Mass Gameplay in Unreal Engine. Unreal Engine Documentation, 2026c. URL https://dev.epicgames.com/documentation/en-us/unreal-engine/overview-of-mass-gameplay-in-unreal-engine. Accessed 2026-10-08.

Epic Games. Navigation System in Unreal Engine. Unreal Engine Documentation, 2026d. URL https://dev.epicgames.com/documentation/en-us/unreal-engine/navigation-system-in-unreal-engine. Accessed 2026-10-08.

Epic Games. Smart Objects in Unreal Engine. Unreal Engine Documentation, 2026e. URL https://dev.epicgames.com/documentation/en-us/unreal-engine/smart-objects-in-unreal-engine. Accessed 2026-10-08.

Epic Games. State Tree in Unreal Engine. Unreal Engine Documentation, 2026f. URL https://dev.epicgames.com/documentation/en-us/unreal-engine/state-tree-in-unreal-engine. Accessed 2026-10-08.

Epic Games. ZoneGraph plugin API reference. Unreal Engine Documentation, 2026g. URL https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Plugins/ZoneGraph. Accessed 2026-10-08.

Christer Ericson. *Real-Time Collision Detection*. CRC Press, 2004. doi: 10.1201/b14581.

Kenneth D. Forbus and Will Wright. Some notes on programming objects in The Sims. Qualitative Reasoning Group, Northwestern University, 2001. URL https://qrg.northwestern.edu/papers/Files/Programming_Objects_in_The_Sims.pdf. Version of 31 May 2001. Accessed 2026-10-08.

John J. Fruin. *Pedestrian Planning and Design*. Metropolitan Association of Urban Designers and

Environmental Planners, New York, 1971.

David “Rez” Graham. An introduction to utility theory. In Steve Rabin, editor, *Game AI Pro 360: Guide to Architecture*, pages 67–80. CRC Press, Boca Raton, FL, 2019. doi: 10.1201/9780429055058-6.

Volker Grimm, Uta Berger, Finn Bastiansen, et al. A standard protocol for describing individual-based and agent-based models. *Ecological Modelling*, 198(1–2):115–126, 2006. doi: 10.1016/j.ecolmodel.2006.04.023.

Volker Grimm, Steven F. Railsback, Christian E. Vincenot, et al. The ODD protocol for describing agent-based and other simulation models: A second update to improve clarity, replication, and structural realism. *Journal of Artificial Societies and Social Simulation*, 23(2):7, 2020. doi: 10.18564/jasss.4259.

Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.

Christopher Hartman and Bedřich Beneš. Autonomous boids. *Computer Animation and Virtual Worlds*, 17(3–4):199–206, 2006. doi: 10.1002/cav.123.

Dirk Helbing and Péter Molnár. Social force model for pedestrian dynamics. *Physical Review E*, 51(5): 4282–4286, 1995. doi: 10.1103/PhysRevE.51.4282.

Dirk Helbing, Illés Farkas, and Tamás Vicsek. Simulating dynamical features of escape panic. *Nature*, 407(6803):487–490, 2000. doi: 10.1038/35035023.

Dirk Helbing, Lubos Buzna, Anders Johansson, and Torsten Werner. Self-organized pedestrian crowd dynamics: Experiments, simulations, and design solutions. *Transportation Science*, 39(1):1–24, 2005. doi: 10.1287/trsc.1040.0108.

S. P. Hoogendoorn and P. H. L. Bovy. Pedestrian route-choice and activity scheduling theory and models. *Transportation Research Part B: Methodological*, 38(2):169–190, 2004. doi: 10.1016/S0191-2615(03)00007-9.

Andreas Horni, Kai Nagel, and Kay W. Axhausen, editors. *The Multi-Agent Transport Simulation MATSim*. Ubiquity Press, London, 2016. doi: 10.5334/baw.

Roger L. Hughes. A continuum theory for the flow of pedestrians. *Transportation Research Part B: Methodological*, 36(6):507–535, 2002. doi: 10.1016/S0191-2615(01)00015-7.

International Organization for Standardization. ISO 20414:2020 Fire safety engineering – Verification and

validation protocol for building fire evacuation models. ISO, Geneva, 2020. URL https://www.iso.org/standard/78348.html. Accessed 2026-10-08.

Damian Isla. Handling complexity in the Halo 2 AI. Game Developers Conference 2005 Proceedings, published on Gamasutra (now Game Developer), 2005. URL https://www.gamedeveloper.com/programming/gdc-2005-proceeding-handling-complexity-in-the-i-halo-2-i-ai. Accessed 2026-10-08.

Mark Joselli, Erick Baptista Passos, Marcelo Zamith, Esteban Clua, Anselmo Montenegro, and Bruno Feijó. A neighborhood grid data structure for massive 3D crowd simulation on GPU. In *2009 VIII Brazilian Symposium on Games and Digital Entertainment (SBGAMES)*, pages 121–131. IEEE, 2009. doi: 10.1109/SBGAMES.2009.22.

Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, et al. Unity: A general platform for intelligent agents. arXiv preprint arXiv:1809.02627, 2018.

JuPedSim developers. JuPedSim: An open-source pedestrian dynamics simulator. Project website, 2026. URL https://www.jupedsim.org/. Accessed 2026-10-24.

Marcelo Kallmann and Daniel Thalmann. Modeling objects for interaction tasks. In Bruno Arnaldi and Gérard Hégron, editors, *Computer Animation and Simulation '98*, Eurographics, pages 73–86. Springer, Vienna, 1999. doi: 10.1007/978-3-7091-6375-7_6.

Mubbasir Kapadia, Nuria Pelechano, Jan Allbeck, and Norman Badler. *Virtual Crowds: Steps Toward Behavioral Realism*. Synthesis Lectures on Visual Computing. Morgan & Claypool, San Rafael, CA, USA, 2016. doi: 10.2200/S00673ED1V01Y201509CGR020.

Ioannis Karamouzas, Brian Skinner, and Stephen J. Guy. Universal power law governing pedestrian interactions. *Physical Review Letters*, 113(23):238701, 2014. doi: 10.1103/PhysRevLett.113.238701.

David G. Kendall. Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded Markov chain. *The Annals of Mathematical Statistics*, 24(3):338–354, 1953. doi: 10.1214/aoms/1177728975.

Ansgar Kirchner and Andreas Schadschneider. Simulation of evacuation processes using a bionics-inspired cellular automaton model for pedestrian dynamics. *Physica A: Statistical Mechanics and its Applications*, 312(1–2):260–276, 2002. doi: 10.1016/S0378-4371(02)00857-9.

Benedikt Kleinmeier, Benedikt Zönnchen, Marion Gödel, and Gerta Köster. Vadere: An open-source

simulation framework to promote interdisciplinary understanding. *Collective Dynamics*, 4, 2019. doi: 10.17815/cd.2019.21.

Erica D. Kuligowski, Richard D. Peacock, and Bryan L. Hoskins. A review of building evacuation models, 2nd edition. NIST Technical Note 1680, National Institute of Standards and Technology, Gaithersburg, MD, 2010.

Averill M. Law. *Simulation Modeling and Analysis*. McGraw-Hill Education, New York, NY, fifth edition, 2015.

Jaedong Lee, Jungdam Won, and Jehee Lee. Crowd simulation by deep reinforcement learning. In *Proceedings of the 11th Annual International Conference on Motion, Interaction, and Games (MIG '18)*, pages 1–7, New York, NY, USA, 2018. ACM. doi: 10.1145/3274247.3274510.

S. Lemercier, A. Jelic, R. Kulpa, et al. Realistic following behaviors for crowd simulation. *Computer Graphics Forum*, 31(2pt2):489–498, 2012. doi: 10.1111/j.1467-8659.2012.03028.x.

Alon Lerner, Yiorgos Chrysanthou, and Dani Lischinski. Crowds by example. *Computer Graphics Forum*, 26(3):655–664, 2007. doi: 10.1111/j.1467-8659.2007.01089.x.

John D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961. doi: 10.1287/opre.9.3.383.

Sean Luke, Claudio Cioffi-Revilla, Liviu Panait, Keith Sullivan, and Gabriel Balan. MASON: A multiagent simulation environment. *SIMULATION*, 81(7):517–527, 2005. doi: 10.1177/0037549705058073.

C. M. Macal and M. J. North. Tutorial on agent-based modelling and simulation. *Journal of Simulation*, 4 (3):151–162, 2010. doi: 10.1057/jos.2010.3.

Ian Millington. *AI for Games*. CRC Press, Boca Raton, FL, third edition, 2019. doi: 10.1201/9781351053303.

Mikko Mononen. Recast Navigation: Navigation-mesh toolset for games. GitHub repository, zlib license, 2009. URL https://github.com/recastnavigation/recastnavigation. Accessed 2026-10-08.

Mehdi Moussaïd, Dirk Helbing, and Guy Theraulaz. How simple rules determine pedestrian behavior and crowd disasters. *Proceedings of the National Academy of Sciences*, 108(17):6884–6888, 2011. doi: 10.1073/pnas.1016507108.

Mehdi Moussaïd, Mubbasir Kapadia, Tyler Thrash, Robert W. Sumner, Markus Gross, Dirk Helbing, and Christoph Hölscher. Crowd behaviour during high-stress evacuations in an immersive virtual

environment. *Journal of the Royal Society Interface*, 13(122):20160414, 2016. doi: 10.1098/rsif.2016.0414.

Soraia Raupp Musse and Daniel Thalmann. Hierarchical model for real time simulation of virtual human crowds. *IEEE Transactions on Visualization and Computer Graphics*, 7(2):152–164, 2001. doi: 10.1109/2945.928167.

Soraia Raupp Musse, Vinicius Jurinic Cassol, and Daniel Thalmann. A history of crowd simulation: The past, evolution, and new perspectives. *The Visual Computer*, 37(12):3077–3092, 2021. doi: 10.1007/s00371-021-02252-w.

Michael J. North, Nicholson T. Collier, Jonathan Ozik, Eric R. Tatara, Charles M. Macal, Mark Bragen, and Pam Sydelko. Complex adaptive systems modeling with Repast Simphony. *Complex Adaptive Systems Modeling*, 1:3, 2013. doi: 10.1186/2194-3206-1-3.

Oasys. MassMotion: Pedestrian simulation and crowd modelling software. Product website, 2026. URL https://www.oasys-software.com/products/massmotion/. Accessed 2026-10-24.

Jeff Orkin. Three states and a plan: The A.I. of F.E.A.R. In *Game Developers Conference 2006*, San Jose, CA, USA, 2006. URL https://gdcvault.com/play/1013282/Three-States-and-a-Plan.

John D. Owens, David Luebke, Naga Govindaraju, Mark Harris, Jens Krüger, Aaron E. Lefohn, and Timothy J. Purcell. A survey of general-purpose computation on graphics hardware. *Computer Graphics Forum*, 26(1):80–113, 2007. doi: 10.1111/j.1467-8659.2007.01012.x.

Sébastien Paris and Stéphane Donikian. Activity-driven populace: A cognitive approach to crowd simulation. *IEEE Computer Graphics and Applications*, 29(4):34–43, 2009. doi: 10.1109/MCG.2009.58.

N. Pelechano, J. M. Allbeck, and N. I. Badler. Controlling individual agents in high-density crowd simulation. In *Proceedings of the 2007 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '07)*, pages 99–108. The Eurographics Association, 2007. doi: 10.2312/SCA/SCA07/099-108.

Nuria Pelechano, Jan M. Allbeck, and Norman I. Badler. *Virtual Crowds: Methods, Simulation, and Control*. Synthesis Lectures on Computer Graphics and Animation. Morgan & Claypool, San Rafael, CA, USA, 2008. doi: 10.2200/S00123ED1V01Y200808CGR008.

Julien Pettré, Jan Ondřej, Anne-Hélène Olivier, Armel Cretual, and Stéphane Donikian. Experiment-based modeling, simulation and validation of interactions between virtual walkers. In *Proceedings of the*

2009 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '09), pages 189–198, New York, NY, USA, 2009. ACM. doi: 10.1145/1599470.1599495.

Craig Reynolds. Big fast crowds on PS3. In *Proceedings of the 2006 ACM SIGGRAPH Symposium on Videogames (Sandbox '06)*, pages 113–121, New York, NY, USA, 2006. ACM. doi: 10.1145/1183316.1183333.

Craig W. Reynolds. Flocks, herds and schools: A distributed behavioral model. *ACM SIGGRAPH Computer Graphics*, 21(4):25–34, 1987. doi: 10.1145/37402.37406.

Craig W. Reynolds. Steering behaviors for autonomous characters. In *Proceedings of the Game Developers Conference 1999*, San Jose, CA, USA, 1999. Miller Freeman Game Group. URL https://www.red3d.com/cwr/steer/gdc99/.

Brian Ricks, Andrew Dobson, Athanasios Krontiris, Kostas Bekris, Mubbasir Kapadia, and Fred Roberts. Generation of crowd arrival and destination locations/times in complex transit facilities. *The Visual Computer*, 36(8):1651–1661, 2020. doi: 10.1007/s00371-019-01761-z.

RiMEA e.V. Richtlinie für Mikroskopische Entfluchtungsanalysen (RiMEA) / Guideline for Microscopic Evacuation Analysis, version 4.1. RiMEA e.V., 2025. URL https://rimea.de/de/rimea-projekt/richtlinie-fuer-mikroskopische-entfluchtungsanalysen/. Versions 4.1.0 (German) and 4.1.1 (German/English). Accessed 2026-10-08.

R. A. Saeed, Diego Reforgiato Recupero, and Paolo Remagnino. Simulating crowd behaviour combining both microscopic and macroscopic rules. *Information Sciences*, 583:137–158, 2022. doi: 10.1016/j.ins.2021.11.028.

Andreas Schadschneider, Wolfram Klingsch, Hubert Klüpfel, Tobias Kretz, Christian Rogsch, and Armin Seyfried. Evacuation dynamics: Empirical results, modeling and applications. In Robert A. Meyers, editor, *Encyclopedia of Complexity and Systems Science*, pages 3142–3176. Springer, New York, NY, 2009. doi: 10.1007/978-0-387-30440-3_187.

Armin Seyfried, Bernhard Steffen, Wolfram Klingsch, and Maik Boltes. The fundamental diagram of pedestrian movement revisited. *Journal of Statistical Mechanics: Theory and Experiment*, 2005(10): P10002, 2005. doi: 10.1088/1742-5468/2005/10/P10002.

Wei Shao and Demetri Terzopoulos. Autonomous pedestrians. *Graphical Models*, 69(5–6):246–274, 2007. doi: 10.1016/j.gmod.2007.09.001.

John F. Shortle, James M. Thompson, Donald Gross, and Carl M. Harris. *Fundamentals of Queueing Theory*. Wiley Series in Probability and Statistics. Wiley, Hoboken, NJ, fifth edition, 2018. doi:

10.1002/9781119453765.

Shawn Singh, Mubbasir Kapadia, Petros Faloutsos, and Glenn Reinman. SteerBench: A benchmark suite for evaluating steering behaviors. *Computer Animation and Virtual Worlds*, 20(5–6):533–548, 2009. doi: 10.1002/cav.277.

Greg Snook. Simplified 3D movement and pathfinding using navigation meshes. In Mark DeLoura, editor, *Game Programming Gems*, pages 288–304. Charles River Media, Rockland, MA, 2000.

Mankyu Sung, Michael Gleicher, and Stephen Chenney. Scalable behaviors for crowd simulation. *Computer Graphics Forum*, 23(3):519–528, 2004. doi: 10.1111/j.1467-8659.2004.00783.x.

Daniel Thalmann and Soraia Raupp Musse. *Crowd Simulation*. Springer, London, second edition, 2013. doi: 10.1007/978-1-4471-4450-2.

The AnyLogic Company. Pedestrian Library. AnyLogic website, 2026. URL https://www.anylogic.com/features/libraries/pedestrian-library/. Accessed 2026-10-09.

Thunderhead Engineering. Pathfinder: Crowd movement simulation and egress software. Product website, 2026. URL https://www.thunderheadeng.com/pathfinder. Accessed 2026-10-24.

Adrien Treuille, Seth Cooper, and Zoran Popović. Continuum crowds. *ACM Transactions on Graphics*, 25 (3):1160–1168, 2006. doi: 10.1145/1141911.1142008.

Jur van den Berg, Ming Lin, and Dinesh Manocha. Reciprocal velocity obstacles for real-time multi-agent navigation. In *2008 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1928–1935. IEEE, 2008. doi: 10.1109/ROBOT.2008.4543489.

Jur van den Berg, Stephen J. Guy, Ming Lin, and Dinesh Manocha. Reciprocal n-body collision avoidance. In Cédric Pradalier, Roland Siegwart, and Gerhard Hirzinger, editors, *Robotics Research*, volume 70 of *Springer Tracts in Advanced Robotics*, pages 3–19. Springer, Berlin, Heidelberg, 2011. doi: 10.1007/978-3-642-19457-3_1.

H. Van Dyke Parunak, Robert Savit, and Rick L. Riolo. Agent-based modeling vs. equation-based modeling: A case study and users' guide. In *Multi-Agent Systems and Agent-Based Simulation (MABS 1998)*, volume 1534 of *Lecture Notes in Computer Science*, pages 10–25. Springer, Berlin, Heidelberg, 1998. doi: 10.1007/10692956_2.

Wouter van Toll and Julien Pettré. Algorithms for microscopic crowd simulation: Advancements in the 2010s. *Computer Graphics Forum*, 40(2):731–754, 2021. doi: 10.1111/cgf.142664.

Tamás Vicsek, András Czirók, Eshel Ben-Jacob, Inon Cohen, and Ofer Shochet. Novel type of phase transition in a system of self-driven particles. *Physical Review Letters*, 75(6):1226–1229, 1995. doi: 10.1103/PhysRevLett.75.1226.

Ulrich Weidmann. Transporttechnik der Fußgänger: Transporttechnische Eigenschaften des Fußgängerverkehrs, Literaturauswertung. IVT Schriftenreihe 90, Institut für Verkehrsplanung, Transporttechnik, Strassen- und Eisenbahnbau (IVT), ETH Zürich, Zürich, 1993.

Uri Wilensky. NetLogo. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL, 1999. URL http://ccl.northwestern.edu/netlogo/. Computer software. Accessed 2026-10-08.

J. Zhang, W. Klingsch, A. Schadschneider, and A. Seyfried. Transitions in pedestrian fundamental diagrams of straight corridors and T-junctions. *Journal of Statistical Mechanics: Theory and Experiment*, 2011(06):P06004, 2011. doi: 10.1088/1742-5468/2011/06/P06004.

Part III
Appendices

Appendix A
Details of Results

The tables below give the per-run detail behind the summaries of Chapter 5. They are written by the analysis scripts of Appendix B.2 from the same recorded files, and times are in simulated minutes unless stated otherwise.

A.1 Shopping Center

Table A.1 lists every capacity run of the first five seeds and Table A.2 gives the statistics of the 30 runs per setting behind Fig. 22. In the counter and table tests, the shop desks kept the capacity set in the map editor, which allowed two people at one desk.

Stations changed	Places	s1	s2	s3	s4	s5	Mean
Shop desks	1	0.84	0.84	0.84	0.84	0.84	0.84
	2	0.84	0.68	0.85	0.84	0.85	0.81
	3	0.84	0.68	0.85	0.84	0.85	0.81
Food counters	1	0.84	0.68	0.85	0.84	0.85	0.81
	2	1.26	1.26	1.02	1.01	1.51	1.21
	3	0.96	0.75	2.31	1.33	1.41	1.35
Food tables	1	0.84	0.68	0.85	0.84	0.85	0.81
	2	0.51	0.51	0.51	0.51	0.51	0.51
	3	0.51	0.51	0.51	0.51	0.51	0.51

Table A.1: Wait per station visit in Mall A for every capacity run (simulated minutes).

The desk test with one place and Mall A as authored use identical settings and seed labels, and with the fixed time step their five pairs of runs were identical, with every rule, station and trip event at the same simulated time, while in the earlier runs, whose frame times varied, the same pairs had differed. The fixed step does not make every scenario repeatable, though, as a repeated corridor run started its people from other positions, and a repeated Daily Urban Life run with seed 1 dispatched 296 rules and saw 96 walks arrive, against 295 and 90 in the run of Table 10.

Stations changed	Places	Mean	Median	SD	95% interval	p
Shop desks	1	0.83	0.84	0.08	0.80–0.86	–
	2	0.84	0.84	0.18	0.77–0.91	0.888
	3	0.84	0.84	0.18	0.77–0.91	0.888
Food counters	1	0.84	0.84	0.18	0.77–0.91	–
	2	1.35	1.34	0.43	1.18–1.51	$<$0.001
	3	2.43	1.26	3.88	0.98–3.88	$<$0.001
Food tables	1	0.84	0.84	0.18	0.77–0.91	–
	2	0.54	0.51	0.09	0.50–0.57	$<$0.001
	3	0.54	0.51	0.09	0.50–0.57	$<$0.001

Table A.2: Wait per station visit in Mall A by number of places with 30 runs per setting, in simulated minutes, where SD is the standard deviation, the interval is the 95% confidence interval of the mean and p compares each setting with one place, and where three runs with an unreleased reservation make the mean with three counter places invalid (1.25 minutes in the other 27 runs).

A.2 Start Delay

Table A.3 gives the values behind Fig. 23, with the mean of each part of the start delay and the share of it spent queueing.

Scenario	Runs	Started, of station rules	Lateness	Queueing	Walking	Position-ing	Total start delay	Start delay spent queueing (%)
Mall A	50	750/750	0.0	0.7	0.3	0.0	1.0	67
Mall B	5	80/80	0.0	0.0	0.3	0.0	0.4	0
Daily Urban Life	5	410/515	1.2	1.8	2.4	0.0	5.4	34

Table A.3: Mean start delay $s - e = \ell + q + w + p$ of station rules and its four parts in simulated minutes, where the last column gives the share of queueing in the total.

A.3 Evacuation

Table A.4 gives each evacuation run behind Fig. 25, for every population and at both frame steps.

Seconds per frame	People	Run	Passed an exit	Through exit 1	Through exit 2	Peak queue, exits 1/2	In the safe zone	Not in the safe zone	Exit switches
1	50	1	50	25	25	20/20	30	20	0
		2	50	25	25	20/20	31	19	0
		3	50	26	24	22/20	28	22	0
	100	1	100	51	49	25/25	23	77	23
		2	100	49	51	25/25	27	73	40
		3	89	89	0	25/25	22	78	71
	200	1	61	31	30	25/25	27	173	4
		2	75	35	40	25/25	29	171	13
		3	44	2	42	25/25	26	174	6
0.25	50	1	50	25	25	20/20	11	39	0
		2	50	25	25	20/20	10	40	0
		3	50	26	24	21/20	11	39	0
	100	1	100	51	49	25/25	11	89	30
		2	100	49	51	25/25	10	90	14
		3	100	51	49	25/25	10	90	32
	200	1	57	31	26	25/25	11	189	7
		2	66	35	31	25/25	10	190	4
		3	58	26	32	25/25	10	190	9

Table A.4: Evacuation results per run, where the people not in the safe zone include those that passed an exit but could not enter it.

A.4 Scaling

Table A.5 lists every scaling run behind Fig. 26. In all of them the game-thread time was 100% of the frame time after rounding, so the table leaves it out, and the evacuation with 1600 people crashed in the first frame with a render-graph assertion.

People	Run	Frames per second	Mean frame time	95th pct. frame time	Mean GPU time	95th pct. GPU time	Peak memory (GB)
Evacuation							
50	1	62.4	16.0	21.1	6.2	7.4	3.0
	2	62.2	16.1	22.7	6.0	6.7	2.9
100	1	30.9	32.3	51.4	11.7	17.3	3.3
	2	34.6	28.9	47.0	10.3	16.8	3.3
200	1	22.6	44.3	54.1	15.5	17.8	4.0
	2	17.8	56.3	74.3	17.8	20.4	4.0
400	1	9.4	106.8	136.1	20.3	21.5	5.5
	2	10.1	98.8	124.7	20.5	21.2	5.4
800	1	4.1	243.4	281.3	21.4	22.5	8.3
1600†	1	crashed in the first frame					
Daily Urban Life							
50	1	38.2	26.2	29.9	6.3	7.2	4.0
	2	37.0	27.0	30.9	6.4	7.3	4.0
100	1	16.8	59.4	70.4	14.2	15.3	4.3
	2	17.0	58.9	69.9	14.1	15.2	4.3
200	1	7.5	133.5	164.9	20.1	21.7	5.1
	2	7.5	134.0	165.5	20.2	21.6	5.0
400	1	3.0	336.2	414.5	28.7	31.4	6.4
	2	3.1	326.2	373.6	28.5	31.5	6.4
800	1	1.2	842.5	893.0	40.6	41.7	9.3
1600†	1	0.4	2287.1	2833.6	65.2	66.1	15.4

Table A.5: All scaling runs with times in milliseconds, where 95th pct. is the 95th percentile and the runs marked † are not comparable with the others.

Appendix B
Tooling

B.1 Building and Running

The project is built with Unreal Engine 5.4 and MSVC 14.38 (`Build.bat ThesisEditor Win64 Development -CompilerVersion=14.38.33130`), because the installed 14.44 toolset cannot compile the engine headers. Mode, clock start, seed, and scenario options are not command-line flags, so `run_scenario.ps1` copies a per-run configuration (Table 9) over `Saved/Config/SimulationConfig.xml`, starts `UnrealEditor.exe` with the flags of Table B.1, waits for an output file or a time-out, and kills the process. Only the scaling runs were rendered, in Visual mode and real time, while all others used Accurate mode, `-nullrhi` and the fixed time step. The public repository (Armada, 2026) holds the framework on its main branch and, on a separate test branch, the recorder, stress test and screenshot tools, the batch scripts and the run configurations, which are kept apart so that they do not add to the framework itself. Every result of Chapter 5 comes from that test branch, which adds these tools and the adjustments listed in Section 5.2.1 to the main branch, while the recorded run files are not public.

B.2 Scaling Schedules, Analysis and Documents

`make_configs.py` also writes the scaling schedules, in which `Agent_k` receives the rules of `Agent_(k mod 50)` for N from 50 to 1600, and the documents are compiled with Tectonic. The numbers of Chapter 5 and Appendix A come from the scripts in `analysis/`: `analyze.py`, `extra_stats.py`, `capacity_stats.py`, `start_delay.py`, `rush_kinds.py`, `review_experiments.py`, `step_check.py` and `figure_data.py`.

Flag	Effect
`/Game/UseCases/<Map> -game -windowed -ResX= -ResY=`	Open the map as a windowed standalone game.
`-nullrhi`	No rendering (no GPU timings or screenshots).
`-RenderOffScreen`	With `-nullrhi` only: no window is opened.
`-WinX=<px> -WinY=<px>`	Window position, and later batch runs opened the window on a second monitor.
`-nosound -nosplash -unattended -NoVerifyGC -abslog=<file>`	Batch-friendly flags and log location.
`-ini:Engine:[...RecastNavMesh]: RuntimeGeneration=Dynamic`	Runtime navigation mesh generation.
`-ExecCmds="r.VSync 0,t.MaxFPS 0"`	Uncapped frame rate for performance runs.
`-UseFixedTimeStep -FPS=30`	Fixed engine step of $1/30$ s, which Accurate mode's $30\times$ clock turns into one simulated second per frame (unrendered runs).
`-RecordSimTime`	Recorder times and position samples in simulated instead of real seconds (fixed-step runs).
`-RunRecord=<Label>`	Write the recorder files (Table 8).
`-ShotAt=<s1>,<s2>,… -ShotFocus=<Tag> -ShotRadius=<cm> -ShotPitch=<deg>`	Screenshot times (real seconds), focus tag, camera distance and pitch (default 50 degrees).
`-TraceAgent=<Agent_N>`	Log one person's position, waypoint and target every frame.
`-StressAgents=<N> -StressWarmup=<s> -StressSeconds=<s> -StressQuit`	Clone people up to N, warm up, measure, exit (defaults 15 s and 60 s, 30 s and 120 s here).
`-StressHideMeshes -NoBlend -RecordPositions=<s>`	Hide the characters, disable the blend and log person positions.
`thesis.ValidationDebugPanel 1`	Console variable that shows the validation debug panel.

Table B.1: Command-line flags used for the runs.

B.3 Repeating the Runs

The test branch of the public repository (Armada, 2026) contains everything that the runs need, in `Project/QA/runner/`. `make_configs.py` writes every run configuration from the base configurations in `base/`, `full_batch.ps1` runs the behavioral phases in Accurate mode with the fixed time step, and `full_batch_vis.ps1` runs the rendered scaling phases in Visual mode, while the paths to the engine and the project at the top of `run_scenario.ps1` and `lib.ps1` must be set for the local machine. A batch skips every run whose recorder folder already exists, so it can be stopped and resumed, and a file named `PAUSE` next to the scripts pauses it between runs. The full batch of 408 runs took about eight hours on the computer of Section 5.2.1, and the analysis scripts of Appendix B.2 then rebuild every table and plot of Chapter 5 and Appendix A from the recorded files.